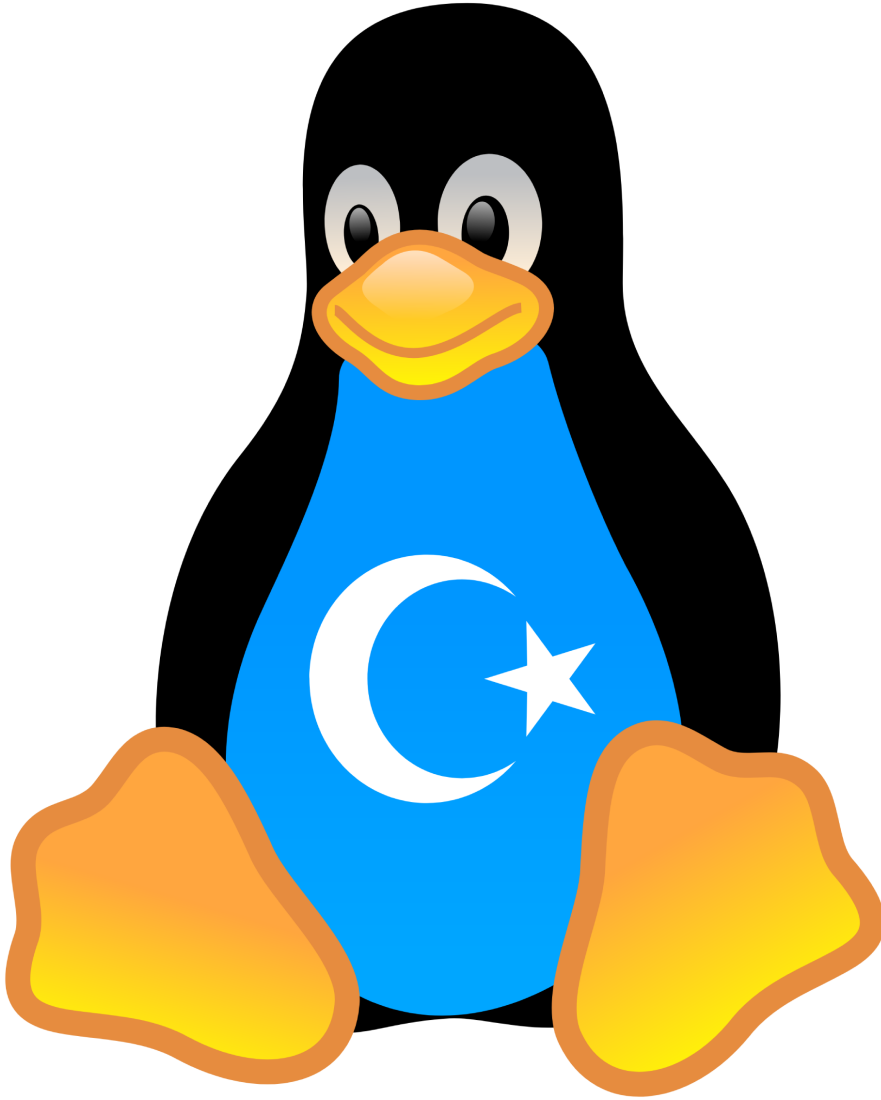


KENDİ LİNUX'UNU YAP



Yazar

- Bayram KARAHAN
- Celalettin AKARSU

Paket Derleme

- Bayram KARAHAN
- Celalettin AKARSU

İletişim

- <https://github.com/kendilinuxunuyap>
- <https://kendilinuxunuyap.github.io>
- kendilinuxunuyap@gmail.com

Ön Söz

Bu kitap, açık kaynak ve özgür yazılım dünyasına ilgi duyan herkes için hazırlanmıştır. Amacı, Türkçe kaynak eksikliğini gidermek ve kendi Linux dağıtımını oluşturmak isteyenlere yol göstermektir.

Bu serinin ilk kitabında, **Temel Linux Sistemi**'nin nasıl derlenip kullanılabilir hale getirileceği anlatılmıştı. Bu kitabımızda ise, oluşturduğumuz **Temel Linux Sistemi** üzerinde **X11 (grafik ortamının)** nasıl derlenip çalıştırılacağı adım adım açıklanacaktır.

Bu kitap, Xorg sunucusunu kaynaktan derlemek isteyen Linux kullanıcıları için kapsamlı bir rehber sunmayı amaçlamaktadır. Xorg, modern Linux sistemlerinde grafiksel kullanıcı arayüzlerinin temelini oluşturan bir bileşendir. Bu rehber, hem yeni başlayanlar hem de deneyimli sistem yöneticileri için tasarlanmıştır ve Debian/Ubuntu tabanlı sistemlere odaklanmaktadır.

Xorg'u kaynaktan derlemek, en son özellikleri kullanmak, özelleştirme yapmak veya belirli bir donanım için optimizasyon sağlamak isteyen kullanıcılar için idealdir. Ayrıca, bu süreç Linux sistemlerinin nasıl çalıştığını anlamak için harika bir öğrenme fırsatı sunar.

İçindekiler

1- giriş	5
3- temelsistem	6
4- GNU Araçlarıyla xorg ve x11 Derleme	9
6- ISO Hazırlama	94
7- Sistem Kurulumu	101
8- sistemcalistirma-inceleme	104
5- paketsistemi	109
9- Yardımcı Konular	120
10- Kaynaklar	140
11- Geliştiricilere Mesajımız	141

Temel Kavramlar

X Pencere Sistemi

X Pencere Sistemi, (X), daha çok GNU/Linux ve Unix benzeri işletim sistemlerinde kullanılan grafik arayüz altyapısıdır. Genel olarak ekrana çizdirme ve kullanıcıdan geribesleme alma işlerini yapan X sunucusunu ve uygulamaların sunucu ile haberleşme için kullanabileceği X kütüphanesini kapsar. Bu sunucu-istemci yapısı sistemin ağ üzerinden çalışmasına da olanak verir. Sunucu doğrudan donanımı yönetebildiği gibi, başka bir altyapı üzerinde de çalışabilir. Bu şekilde diğer işletim sistemlerinde de X sunucusu çalıştırılabilmektedir.

1984 yılında MIT tarafından geliştirilen bu sistem, ağ üzerinden grafiksel uygulamaların çalıştırılmasını mümkün kılar.

İstemci-Sunucu Mimarisi

X, istemci-sunucu mimarisiyle çalışır:

- *İstemciler*, grafik uygulamaları oluşturur.
- *Sunucu*, ekran, klavye ve fare gibi giriş/çıkış aygıtlarını kontrol eder.

Özellikler

X Pencere Sistemi şu özellikleri sunar:

- Çoklu pencere yönetimi
- Ekran paylaşımı
- Uzaktan erişim imkânı

Örneğin, bir X istemcisi, farklı bir bilgisayarda çalışan bir X sunucusuna bağlanarak çalışabilir. Bu sayede kullanıcılar, başka sistemlerdeki uygulamalara kolayca erişebilirler.

X11 Protokolü

Sistemin temelini **X11 protokolü** oluşturur ve bu protokol, geniş bir uygulama yelpazesine destek verir.

Kaynak

[X Pencere Sistemi](#)

Xorg Nedir?

Xorg (veya **X.Org Server**), Unix ve Linux tabanlı işletim sistemlerinde grafiksel kullanıcı arayüzünü (GUI) sağlayan **X Pencere Sistemi (X Window System)**'in en yaygın açık kaynaklı uygulamasıdır. Xorg, ekran, klavye, fare gibi giriş/çıkış aygıtları ile kullanıcı uygulamaları arasında aracı görevi görerek pencere yönetimini ve grafik çıktısı sağlar.

Kısaca Tanımı

Xorg, **X11 protokolünü** temel alarak çalışan bir **görüntü sunucusudur (display server)**. Masaüstü ortamlarının (GNOME, KDE, XFCE vb.) çalışmasını sağlar ama kendisi bir masaüstü ortamı değildir.

Xorg Ne Yapar?

- Grafik kartıyla iletişim kurarak pikselleri ekrana yerleştirir.
- Klavye ve fare gibi giriş aygıtlarını işler.
- Pencereyi yönetmez ama pencere yöneticilerinin çalışmasını sağlar.
- Uzaktaki makinelerden grafik uygulamalarını çalıştırabilir (X forwarding).

Temel Bileşenler

Xorg, aşağıdaki bileşenleri içerebilir veya bunlarla birlikte çalışır:

- **xorg.conf**: Yapılandırma dosyası (modern sistemlerde genellikle otomatik yapılandırma kullanılır).
- **DRI/DRM**: *Direct Rendering Infrastructure / Direct Rendering Manager*; doğrudan donanım erişimi.
- **Mesa**: Açık kaynak OpenGL implementasyonu.
- **Xlib, XCB**: X protokolünü kullanan programlama arabirimleri.

Yapılandırma Dosyaları

- `/etc/X11/xorg.conf` (eski sistemlerde)
- `/etc/X11/xorg.conf.d/` dizini altındaki parçalı yapılandırmalar
- Modern sistemlerde otomatik olarak `/var/log/Xorg.0.log` dosyasına günlük yazar

Kaynaklar

- [X.Org Foundation – Resmi Site](#)
- [X.Org Server GitHub](#)
- [Arch Wiki – Xorg](#)
- [Gentoo Wiki – Xorg Guide](#)

X11 Nedir?

X11 (veya **X Window System, version 11**), Unix benzeri işletim sistemlerinde grafiksel kullanıcı arayüzlerini oluşturmak için kullanılan istemci-sunucu tabanlı bir pencereleme sistemidir. İlk olarak 1984 yılında MIT tarafından geliştirilen X Window System'in 11. versiyonudur ve günümüzde birçok Linux ve Unix sistemde temel grafik altyapısını oluşturur.

X11 Ne İşe Yarar?

- Ekran, klavye, fare gibi aygıtlar ile çalışan uygulamalar arasında aracı olur.
- GUI (grafik kullanıcı arayüzü) oluşturmaz, ama masaüstü ortamlarının GUI'yi oluşturmaya olanak tanır.
- İstemci-sunucu modeli sayesinde uzaktan masaüstü kullanımına izin verir (örneğin ssh -X ile).

Temel Özellikleri

- **Ağ şeffaflığı:** X11, ağ üzerinden çalışan istemcilerin sunucuya bağlanmasını destekler.
- **Donanımdan bağımsızlık:** Grafik donanım soyutlaması sağlar.
- **Uzun ömür:** 1980'lerden beri gelişmiş ve kullanılmaktadır.
- **Modüler yapı:** X sunucusu, pencere yöneticisi ve istemciler ayrı bileşenlerdir.

X11 ile İlgili Bileşenler

- **X Server:** X protokolünü uygulayan grafik sunucusu (örnek: Xorg)
- **X Clients:** Grafik uygulamaları (örnek: terminal, dosya yöneticisi)
- **Window Manager:** Pencere davranışlarını kontrol eden yazılım (örnek: i3, Openbox)
- **Display Manager:** Oturum açmayı yöneten grafik arayüz (örnek: GDM, LightDM)

X11 ile Xorg Arasındaki Fark

- **X11**, bir protokoldür. Grafik uygulamaların nasıl davranması gerektiğini tanımlar.
- **Xorg**, X11 protokolünü uygulayan yazılımdır. Yani X11'in çalışan ve somut hale getirilmiş halidir.

Kaynaklar

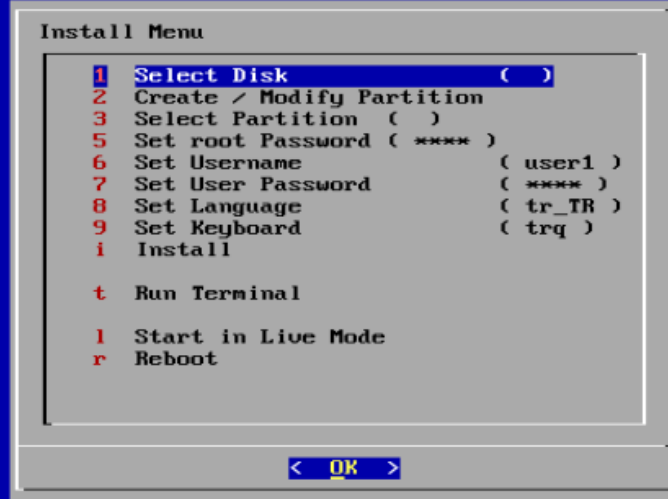
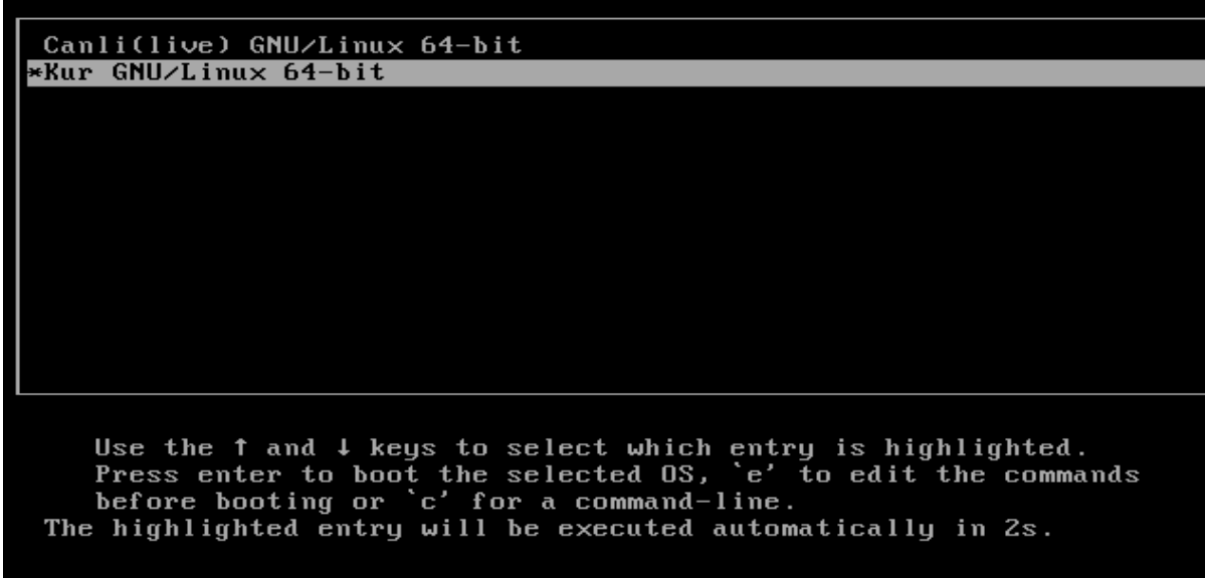
- [X Window System - Wikipedia](#)
- [X.Org Foundation](#)
- [man X \(X11 manual\)](#)
- [Arch Wiki - Xorg](#)

Temel Sistemin Hazırlanması

Bir önceki kitabımızda Temel Linux sistemini oluşturmayı ve iso halinde kurulumu anlatmıştık. Şimdi ise bu temel sistemi kurarak bu sistem üzerine **xorg** derleyerek **x11** pencere sisteminin nasıl çalışacağı anlatılacaktır.

Temel Sistem Kurulumu

Sistemin kurulumu için resimlerde görünen sıraya göre seçimler yapmalıyız.



Kurulum menüsünde kullanıcı adları ve parolaları, klavye varsayılan olarak;

- Kullanıcı: root Parola: 1
- Kullanıcı: user1 Parola: 1
- Dil : tr_TR
- Klavye : trq

menüden değişiklik yapabilirsiniz. Değişiklik yapmadan sadece kurulum diskini ve disk bölümünü seçip Install(Yükle) işlemi yapabilirsiniz.



```
kurulumu geçildi.....
legacy kurulum .....
Kurulum Yapılacak Disk sda
mount: /kaynak: WARNING: source write-protected, mounted read-only.
*****disk bölümleri biçimlendiriliyor *****
e2fsck 1.47.0 (5-Feb-2023)
e2fsck: need terminal for interactive repairs
tune2fs 1.47.0 (5-Feb-2023)
Recovering journal.

This operation requires a freshly checked filesystem.

Please run e2fsck -f on the filesystem.

mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2096896 4k blocks and 524288 inodes
Filesystem UUID: 9ea082d0-a034-4dab-8e2f-564e68c99c9c
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

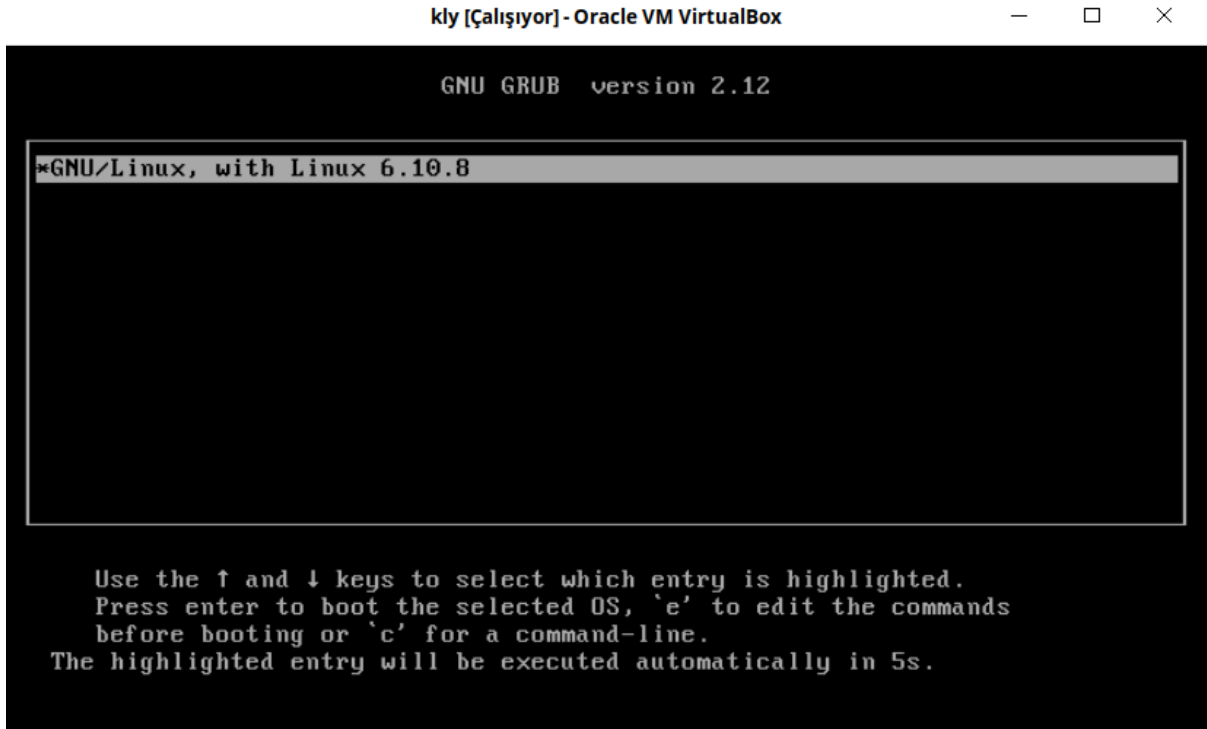
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

Information: You may need to update /etc/fstab.

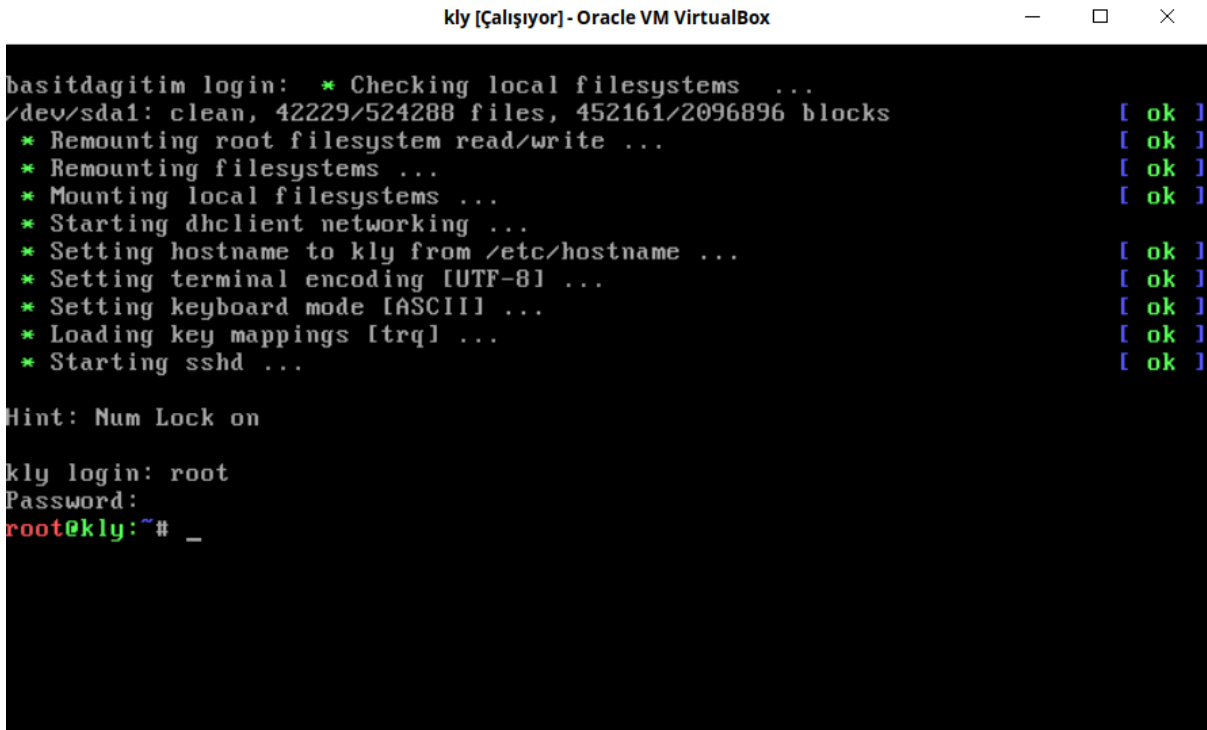
***** kurulum başladı*****
Sistem yüklenmeye başlandı. Tahmini 3-5 dakika sürecektir.. Lütfen bekleyiniz.....
.....
```

Sistemin Çalışması

Sistem kurulumu gerçekleştiğinde sistem resimde görüldüğü gibi açılmalıdır.



Sisteme **root** kullanıcısı olarak giriş yapıldığı görülmektedir.



Temel Sisteme Internet Bağlantısı

Temel Sistem içerisinde bulunan **iproute2**, **dhclient**, **net-tools** paketleri derlendi ve açılışta **dhclient** aktif hale gelmektedir. Bu paket ile ağa dahil olunmaktadır.

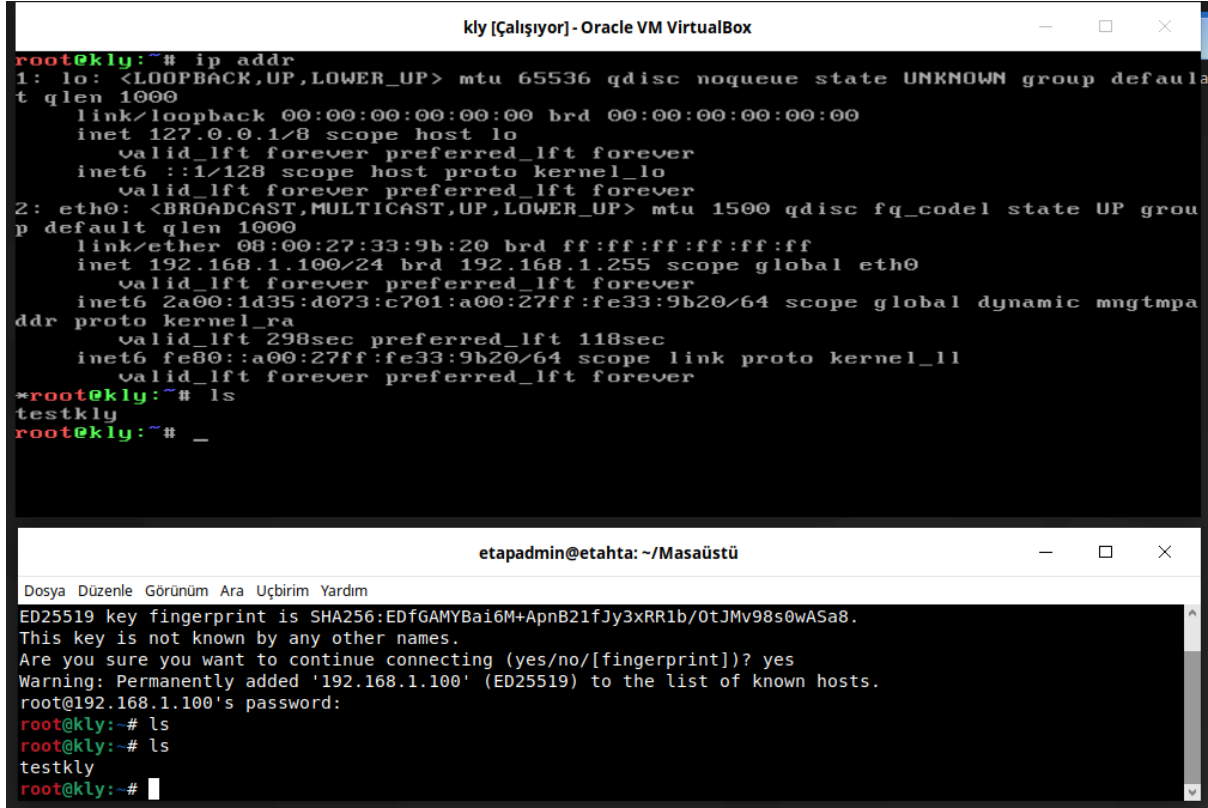
Aşağıda **kly Temel Sistem** ile atanan ip adresini görmekteyiz. Eğer ip adresi almamışsa terminalde **dhclient** komutunu çalıştırınız.

```
kly [Çalışıyor] - Oracle VM VirtualBox
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
        valid_lft 298sec preferred_lft 118sec
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@kly:~# uname -a
Linux kly 6.10.8 #1 SMP PREEMPT_DYNAMIC Sat Sep  7 18:49:23 +03 2024 x86_64 GNU/Linux
root@kly:~#
```

Temel Sisteme openssh ile Bağlanma

ssh terminal üzerinden uzak bilgisayarlara erişim yapan bir uygulamadır. **Temel Sistem** içerisinde ssh paketi derlendi ve açılıştan aktif hale gelmektedir. ssh kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme ssh** ile erişimin nasıl yapıldığı görülmektedir.



The image shows two terminal windows. The top window is titled 'kly [Çalışıyor] - Oracle VM VirtualBox' and shows the output of the 'ip addr' command. It displays details for the loopback interface 'lo' (127.0.0.1) and the ethernet interface 'eth0' (192.168.1.100). The bottom window is titled 'etapadmin@etahta: ~/Masaüstü' and shows the output of the 'ls' command. It displays the output of the 'ls' command, showing the directory structure of the user's home directory.

```
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
*root@kly:~# ls
testkly
root@kly:~# _
```

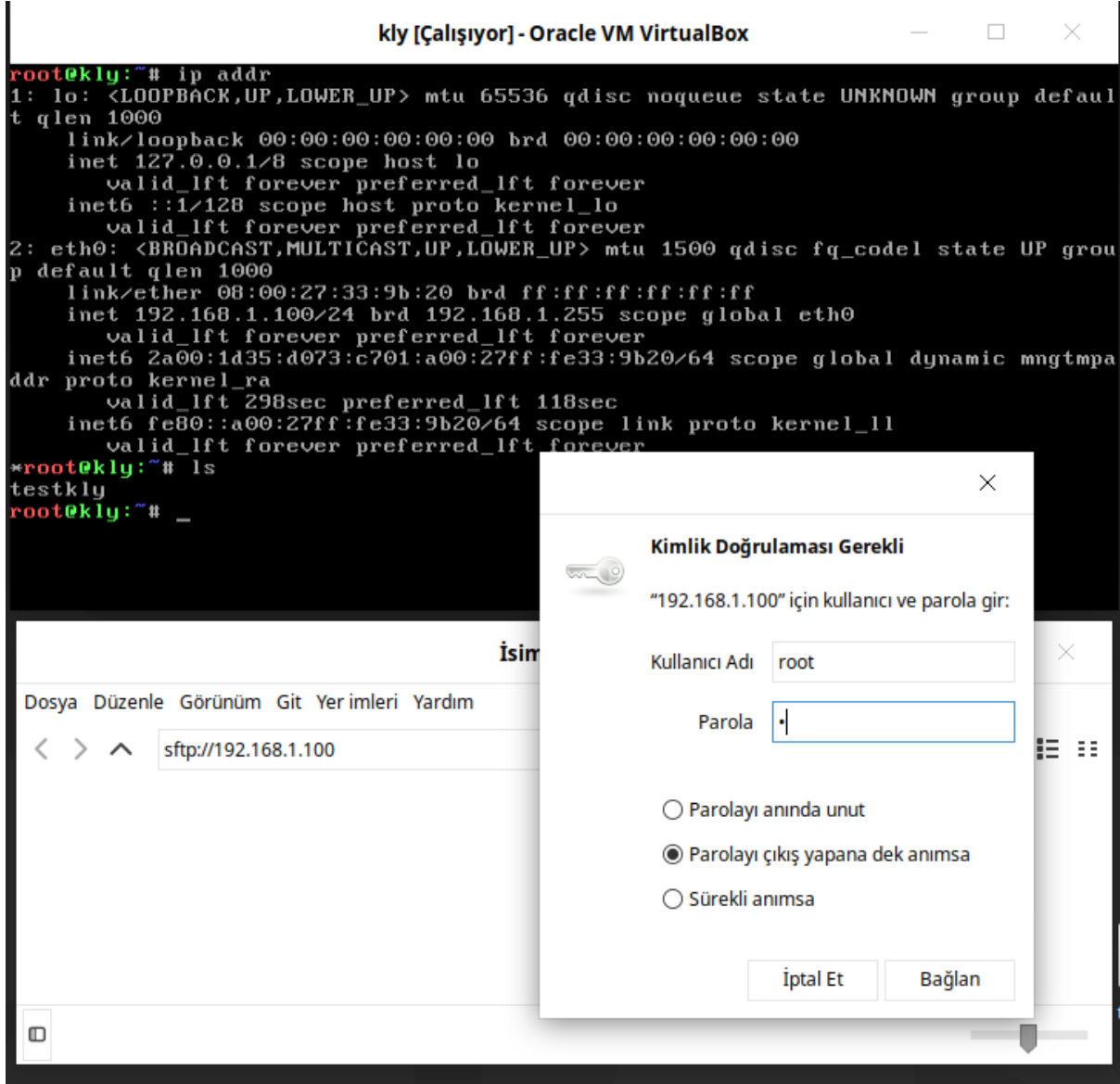
```
etapadmin@etahta: ~/Masaüstü
Dosya Düzenle Görünüm Ara Uçbirim Yardım
ED25519 key fingerprint is SHA256:EDfGAMYBai6M+ApnB21fJy3xRR1b/OtJMv98s0wASa8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.100' (ED25519) to the list of known hosts.
root@192.168.1.100's password:
root@kly:~# ls
testkly
root@kly:~#
```

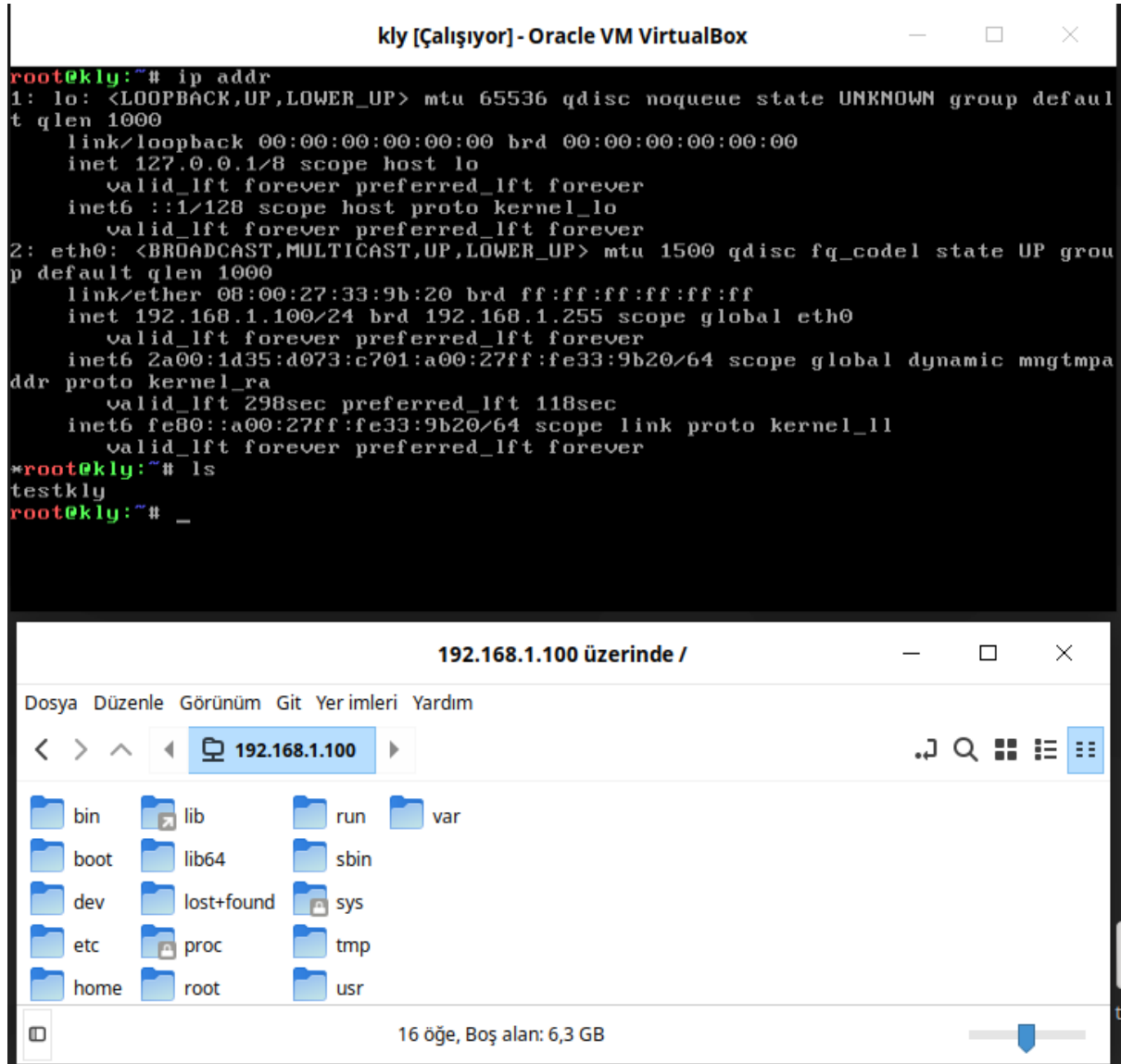
Bu doküman boyunca paketleri **Debian** ortamında derleyeceğiz. Derlediğimiz paketleri **Temel Sistem'e** kopyalacağız. Kopyalama işlemini **ssh** paketi içinde gelen **sftp** veya **scp** ile yapacağız. Kopyalanan paketlerin **kurulması ve kaldırılması** gibi işlemleri **ssh** bağlantısı üzerinden gerçekleştireceğiz.

Temel Sisteme sftp ile Bağlanma

sftp terminal veya pencere yöneticisi üzerinden uzak bilgisayarlara erişim yapan bir uygulamadır. **Temel Sistem** içerisinde **ssh** paketinin bir parçası olarak gelmektedir. sftp kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme sftp** ile erişimin nasıl yapıldığı görülmektedir.

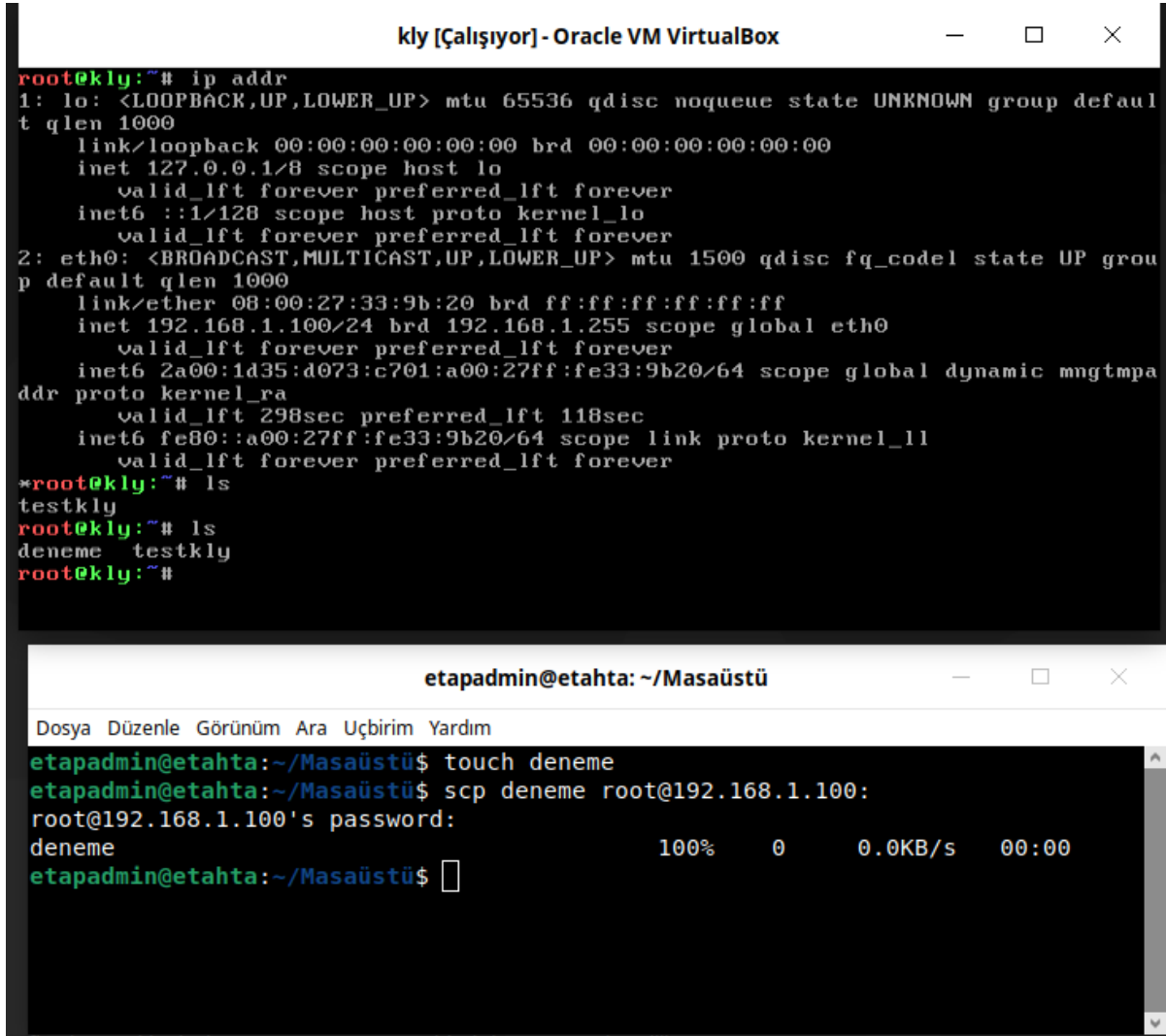




Temel Sisteme scp ile Dosya Kopyalama

scp terminal üzerinden uzak bilgisayarlara dosya kopyalaması yapan bir uygulamadır. **Temel Sistem** içerisinde **ssh** paketinin bir parçası olarak gelmektedir. scp kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme scp** ile **deneme** dosyasının nasıl kopyalandığı görülmektedir.



The image shows two terminal windows. The top window, titled 'kly [Çalışıyor] - Oracle VM VirtualBox', shows the output of the 'ip addr' command for the 'lo' and 'eth0' interfaces. The bottom window, titled 'etapadmin@etahta: ~/Masaüstü', shows the execution of 'touch deneme' and 'scp deneme root@192.168.1.100:' commands, followed by a password prompt and a progress bar for the file transfer.

```
kly [Çalışıyor] - Oracle VM VirtualBox
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
        addr proto kernel_ra
        valid_lft 298sec preferred_lft 118sec
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
*root@kly:~# ls
testkly
root@kly:~# ls
deneme testkly
root@kly:~#

etapadmin@etahta: ~/Masaüstü
Dosya Düzenle Görünüm Ara Uçbirim Yardım
etapadmin@etahta:~/Masaüstü$ touch deneme
etapadmin@etahta:~/Masaüstü$ scp deneme root@192.168.1.100:
root@192.168.1.100's password:
deneme                               100%   0   0.0KB/s   00:00
etapadmin@etahta:~/Masaüstü$
```


Temel Sistem ve Paket Sistemi(kly)

Paket sistemi, sisteme paket (**kurma, kaldırma, index güncelleme**) gibi temel işlemlerin yapılmasını sağlar. Bu temel işlemleri yapacak **Temel Sistem** üzerinde **base-file** paketinden gelen scriptlerimiz yüklü olarak gelmektedir. **Yardımcı Konular** bölümünde paket sisteminin nasıl kullanılacağı anlatılmıştır.

Aşağıda **kly Temel Sistem** üzerinde hazır gelen paket sistemi uygulamalarımız görülmektedir.

kly [Çalışıyor] - Oracle VM VirtualBox

```
root@kly:~# ls -l /bin/kly*
-rwxr-xr-x 1 root root 22588 Jul 21 08:15 /bin/kly
-rwxr-xr-x 1 root root 1224 Nov 17 2024 /bin/klykaldir
-rwxr-xr-x 1 root root 1249 Nov 17 2024 /bin/klykur
-rwxr-xr-x 1 root root 2490 Nov 17 2024 /bin/klypaketle
-rwxr-xr-x 1 root root 252 Nov 17 2024 /bin/klyupdate
root@kly:~# kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c packagedirectory target(default=/)
-i, --install          : package install. use: kly -i packagename target(default=/)
-ri, --reinstall       : package install. use: kly -ri packagename target(default=/)
-pi, --packagefile     : packagefile install. use: kly -pi packagefile target
-r, --remove           : package remove. use: kly -r packagename target(default=/)
-h, --help             : kly help
root@kly:~#
```

- klypaketle: kly paketi oluşturur.
- klykur: kly paketini sisteme kurar.
- klykaldir: Sistemde kurulu olan paketi kaldırır.
- klyupdate: github üzerinden güncel paket listelerini sistemde günceller.

GNU Araçlarıyla xorg ve x11 Derleme konu başlığı altında paketleri **kly Paket Sistemi** kullanarak derleyeceğiz. Derleme işlemini **klypaketle** komutuyla yapacağız. Derlenen paketleri **scp** ve **sftp** kullanarak **Temel Sistem** üzerine kopyalayacağız. **klykur** komutumuzla kopyaladığımız paketi **Temel Sistem** üzerine kuracağız. Oluşturduğumuz paketleri istersek(daha doğru bir yöntem) github'a yükleyip, github üzerinden kurabiliriz. Oluşturulan paketlerin github üzerine nasıl yükleneceği **Yardımcı Konular** bölümünde **Paketlerin Githuba Yüklenmesi** başlığı altında anlatıldı.

kly Paket Sistemi Kullanımı

Paket sistemi, sisteme paket (**kurma, kaldırma, index güncelleme**) gibi temel işlemlerin yapılmasını sağlar. **Temel Sistem** kitabımızda **Paket Sistemi(kly)** tasarımı anlatılmıştı. Bu kitapta ise paketleri derlerken kly paket sistemi kullanılarak derleyeceğiz. Paket sistemi paketleri tekrar tekrar derlemek yerine paket olarak hazırlamakta ve istediğimiz zaman oluşturulan sisteme yüklenmekte veya kaldırılmaktadır. Bir sorun olduğu farkedildiğinde ise tekrar derlenerek sadece bu paket güncelenebilmektedir. Bu tür avantajlardan dolayı paket sistemini kullanacağız.

Burada paket sisteminin nasıl kullanılacağı anlatılacaktır. Paket sistemimiz **Temel Sistem** üzerinde **base-file** paketiyle sisteme yüklendi. Aşağıda **kly Temel Sistem** üzerinde hazır gelen paket sistemi uygulamalarımız görülmektedir.

kly [Çalışıyor] - Oracle VM VirtualBox

```
root@kly:~# ls -l /bin/kly*
-rwxr-xr-x 1 root root 22588 Jul 21 08:15 /bin/kly
-rwxr-xr-x 1 root root 1224 Nov 17 2024 /bin/klykaldir
-rwxr-xr-x 1 root root 1249 Nov 17 2024 /bin/klykur
-rwxr-xr-x 1 root root 2490 Nov 17 2024 /bin/klypaketle
-rwxr-xr-x 1 root root 252 Nov 17 2024 /bin/klyupdate
root@kly:~# kly --help
Usage: kly <options>
-u, --update      : package index update. use: kly -u
-c, --create      : create kly package. use: kly -c      packagedirectory target(default=/)
-i, --install     : package install. use: kly -i      packagename target(default=/)
-ri, --reinstall  : package install. use: kly -ri     packagename target(default=/)
-pi, --packagefile : packagefile install. use: kly -pi  packagefile target
-r, --remove      : package remove. use: kly -r      packagename target(default=/)
-h, --help        : kly help
```

kly: *klypaketle, klykur, klykadir, klyupdate* scripplerimizin hepsini tek scriptte toplanmış halidir. Ayrıca paketlerin kurulumu ve kaldırılması sırasında bağımlılıklarını da paketle birlikte kurmakta veya kaldırmakta. Bu dokümanda paketlerin derlenmesi, kurulması, kaldırılması vb. işlemler **kly** scripti kullanılarak yapılacaktır. **klypaketle, klykur, klykaldir, klyupdate** scripleri bir öğretici olması nedeniyle bulunmaktadır. Daha kapsamlı işlemler için yetersiz kalacaktır.

kly Paket Sisteminin Debian'a Kurulumu

kly paket sistemi **Temel Sistem** üzerinde kurulu olarak gelmektedir. Üstte verilen resimde görülmektedir. Kullanımı ve parametrelerini unutmanız durumunda **kly --help** komutuyla kullanımı öğrenebilirsiniz.

Debian üzerinde kullanmak için kly scriptimizi indirip **/bin** konumuna kopyalayalım. Aşağıda verilen komutları sırasıyla çalıştırınız. **kly** paket sisteminin sorunsuz çalışması için **busybox** paketinin kurulu olması gerekmektedir. Sorunsuz çalışmışsa **Debian** sistemimize **kly** paket sistemimiz kurulmuştur.

```
wget https://kendilinuxunuyap.github.io/_static/files/base-file/kly
sudo mv kly /bin/kly
sudo chown root:root /bin/kly
sudo chmod 755 /bin/kly
sudo apt update
sudo apt install busybox-static -y
```

kly ile Paket Yapma

GNU Araçlarıyla **xorg ve x11** Derleme konu başlığı altında paketleri kly Paket Sistemi kullanılarak derleyeceğiz. Derleme işlemini **kly -c** komutuyla yapacağız. Derlenen paketleri **scp** ve **sftp** kullanarak **Temel Sistem** üzerine kopyalayacağız. **kly -i** komutumuzla kopyaladığımız paketi **Temel Sistem** üzerine kuracağız. Oluşturduğumuz paketleri istersek(daha doğru bir yöntem) github'a yükleyip. github üzerinden kururabiliriz.

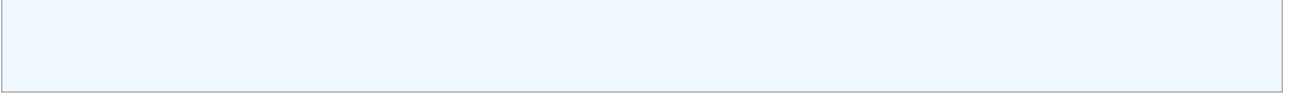
kly ile Paket Kur



kly ile Paket Kaldır



kly Paketlerini Githuba Yükleme ve İndexleme



kly ile Paket Listelerini Güncelleme

--

GNU Araçlarıyla xorg ve x11 Derleme

Kurulum Dizini: Dağıtımın Oluşacağı Yer

Paket derleme işlemi öncesi aşağıdaki konuları bilmemiz gerekmektedir. Bunlar;

1. Derleme(Dinamik/Static)
2. chroot Kullanımı
3. İso Oluşturma
4. ssh Kullanımı
5. sftp Kullanımı
6. scp Kullanımı
7. VirtualBox Kullanımı
8. cfdisk Kullanımı

Burada liste halinde verilen konu başlıkları bu dokümanın **Yardımcı Konular** bölümünde anlatılmaktadır.

Bundan sonraki adımlarda kendi dağıtımımızın **xorg ve x11** pencere sistemini derleyerek **Temel Sistem** üzerinde çalıştıracağız!

xorg ve x11 için ihtiyacımız olan Paketler

0- base-file	25- elfutils	50- popt
1- glibc	26- libselinux	51- icu
2- readline	27- tar	52- iproute2
3- ncurses	28- zlib	53- net-tools
4- bash	29- brotli	54- dhcp
5- openssl	30- curl	55- openrc
6- acl	31- shadow	56- rsync
7- attr	32- file	57- kbd
8- libcap	33- eudev	58- kernel
9- libpcre2	34- cpio	59- dialog
10- gmp	35- libsepol	60- live-boot
11- coreutils	36- kmod	61- live-config
12- util-linux	37- audit	62- parted
13- grep	38- libxcrypt	63- busybox
14- sed	39- libnsl	64- nano
15- mpfr	40- libbsd	65- grub
16- gawk	41- libtirpc	66- efibootmgr
17- findutils	42- e2fsprogs	67- efivar
18- gcc	43- dosfstools	68- libssh
19- libcap-ng	44- initramfs-tools	69- openssh
20- sqlite	45- libxml2	70- pam
21- gzip	46- expat	71-
22- xz-utils	47- libmd	72-
23- zstd	48- libaio	73-
24- bzip2	49- lvm2	74-

Bağımlılık Zinciri

Linux paketinin sorunsuz çalışabilmesi için bağımlı olduğu tüm paketlerin önceden derlenmiş olması gerekir. **x11**'in en temel paketleri **xorg-server**, **mesa**, **llvm**, **cairo** paketleridir. Tüm paketleri derlese bile **xorg-server**, **mesa**, **llvm**, **cairo** paketleri düzgün ve uyumlu versiyonları olmadığı zaman x penceremiz açılmayacaktır. Buradaki tüm paketler ve bağımlılıkları derlendikten sonra **Xorg:0** şeklinde x pencere sistemimiz çalışacaktır.

Hazırlık: Gerekli Derleme Araçlarını Kurun!

Paket derleme işlemine başlamadan önce, aşağıdaki temel araçları sisteminize kurmalısınız.

```
sudo apt update
sudo apt-get install debootstrap xorriso mtools make squashfs-tools gcc wget unzip xz-utils tar zstd fakeroot \
autoconf automake autotools-dev make meson cmake ninja-build pkgconf patch libtool grub-pc grub-pc-bin
```

xorg-server

ISO Hazırlama

Dağıtıma özgü ISO oluşturmak için o dağıtımın paket sistemi kullanılır. Bu dokümanda anlatılan **kly** paket sistemi, önceki bölümde detaylıca açıklanmıştır.

Bu bölümde, **kly** paket sistemini kullanarak ISO oluşturma adımları basit ve anlaşılır şekilde verilecektir. Aşağıda paylaşılan script ile hazırlanan iso **/tmp/distro/distro.iso** konumunda olacaktır. Sistemi oluşturan paketleri **github.com/kendilinuxunuyap/kly-binary-packages** konumundan indirilerek oluşturmaktadır.

Not: Anlatılan yöntem genel Linux dağıtımlarının ISO oluşturma mantığına dayanır. Araçlar, dizinler ve komutlarda küçük farklılıklar olabilir, ancak iş akışı aynıdır.

kly Paket Sistemiyle ISO Oluşturma Scripti

```
#-----
#!/bin/bash
rootfs="/tmp/distro/rootfs"
distro="/tmp/distro"
rm -rf $rootfs
mkdir -p $rootfs -p $rootfs/bin $rootfs/etc
##Kurulum scripti
cp -prf files/bin/* $rootfs/bin/
cp -prf files/etc/* $rootfs/etc/

# temel izinler ve dosyalar oluşturuluyor
cd $rootfs/
mkdir -p bin dev etc home lib64 proc root run/sbin sys usr var etc/kly \
tmp tmp/kly/kur var/log var/tmp usr/lib64/x86_64-linux-gnu \
usr/lib64/pkgconfig usr/local/{bin,etc,games,include,lib,sbin,share,src}
ln -s lib64 lib
cd var&&ln -s ../run run&&cd -
cd usr&&ln -s lib64 lib&&cd -
cd usr/lib64/x86_64-linux-gnu&&ln -s ../pkgconfig pkgconfig&&cd -

echo -e "/bin/sh\n/bin/bash\n/bin/rbash\n/bin/dash" >> "$rootfs/etc/shell"
echo "tmpfs /tmp tmpfs rw,nodev,nosuid 0 0" >> "$rootfs/etc/fstab"
echo "127.0.0.1 kly" >> "$rootfs/etc/hosts"
echo "kly" > "$rootfs/etc/hostname"
echo "nameserver 8.8.8.8" > "$rootfs/etc/resolv.conf"

# paket adresi ekleniyor
echo "kendilinuxunuyap/kly-binary-packages">$rootfs/etc/kly/sources.list

### installing kly create_package in rootfs
echo root:x:0:0:root:/root:/bin/sh > $rootfs/etc/passwd
chmod 755 $rootfs/etc/passwd

### system chroot bind/mount
for dir in dev dev/pts proc sys; do mount -o bind /$dir $rootfs/$dir; done
## paket listesi güncelleniyor
$rootfs/bin/klyupdate $rootfs

## paketler kuruluyor
for paket in glibc readline ncurses \bash openssl acl attr libcap libpcrc2 gmp \
coreutils util-linux \grep \sed mpfr \gawk findutils libgcc libcap-ng sqlite \
\gzip xz-utils zstd \bzip2 \elfutils libselinux \tar \zlib brotli curl shadow \
\file eudev cpio libsepol kmod audit libxcrypt libnsl pam libtirpc e2fsprogs \
dosfstools initramfs-tools libxml2 expat libmd libaio lvm2 popt icu iproute2 \
net-tools dhcp openrc rsync kbd busybox kernel kernel-headers live-boot \
live-config parted nano grub dialog efibootmgr efivar libssh openssl
do
chroot $rootfs /bin/klykur $paket;
done

#### system chroot umount
for dir in dev dev/pts proc sys ; do while umount -lf -R $rootfs/$dir \
2>/dev/null ; do true; done done
exit
```

```

#-----
# scriptin devamı
chroot $rootfs useradd live -m -s /bin/sh -d /home/live
chroot $rootfs echo -e "live\nlive\n"|chroot $rootfs passwd live

for grp in users tty wheel cdrom audio dip video plugdev netdev; do
    chroot $rootfs usermod -aG $grp live || true
done

sed -i "/agetty_options/d" $rootfs/etc/conf.d/agetty
echo -e "\nagetty_options=\"-l /usr/bin/login\"" >> $rootfs/etc/conf.d/agetty

### update-initrd
fname=$(basename $rootfs/boot/config*)
kversion=${fname:7}
mv $rootfs/boot/config* $rootfs/boot/config-$kversion
cp $rootfs/boot/config-$kversion $rootfs/etc/kernel-config

chroot $rootfs update-initramfs -u -k $kversion
#### system chroot umount
for dir in dev dev/pts proc sys ;
do while umount -lf -R $rootfs/$dir 2>/dev/null ; do true; done done

mkdir -p "$distro/iso" "$distro/iso/boot" "$distro/iso/boot/grub"
mkdir -p "$distro/iso/live"
#### Copy kernel and initramfs
cp -pf $rootfs/boot/initrd.img-* $distro/iso/boot/initrd.img
cp -pf $rootfs/boot/vmlinuz-* $distro/iso/boot/vmlinuz
rm -rf $rootfs/boot

#### Create squashfs
mksquashfs $rootfs $distro/filesystem.squashfs -comp xz -wildcards
mv $distro/filesystem.squashfs $distro/iso/live/filesystem.squashfs

#### Write grub.cfg
cat << EOF > "$distro/iso/boot/grub/grub.cfg"
set timeout=3 # Timeout for menu
set default=1 # Default boot entry

# Menu Colours
set menu_color_normal=white/black
set menu_color_highlight=white/blue
insmod all_video
terminal_output console
terminal_input console
menuentry "Canli(live) GNU/Linux 64-bit" --class liveiso {
    linux /boot/vmlinuz boot=live init=/sbin/openrc-init net.ifnames=0 \
    biosdevname=0
    initrd /boot/initrd.img
}
menuentry "Kur GNU/Linux 64-bit" --class liveiso {
    linux /boot/vmlinuz boot=live init=/bin/kur quiet
    initrd /boot/initrd.img
}
EOF
grub-mkrescue $distro/iso/ -o $distro/distro.iso

```

Kaynaklar:

- <https://www.subrat.info/build-kernel-and-userspace/> - 08/07/2025
- <https://medium.com/@chienhaotan/compiling-and-running-a-minimal-kernel-with-busybox-bfc45a991017> - 08/07/2025
- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)) - 08/07/2025
- <https://chatgpt.com/share/6875084c-6050-8012-9229-a37b47351aa2> - 14/07/2025

Sistem Kurulumu

Sistem Kurma

Hazırlanan ISO ile birlikte, farklı kurulum araçları da gelebilir. Bu araçlar, çeşitli kurulum yöntemlerini destekleyebilir. En sık kullanılan kurulum yöntemleri şunlardır:

1. Tek bölüme sistem kurulumu
2. UEFI sistem kurulumu (boot + sistem)

Bu bölümde, tek bölüm kurulum ve boot + sistem şeklinde iki farklı kurulum yöntemi sırayla anlatılacaktır. Anlatılan yöntemler, farklı kullanıcı senaryolarına cevap verebilecek şekilde tasarlanmıştır.

Ancak dikkat edilmesi gereken önemli bir nokta şudur: Her yöntemi ayrı ayrı son kullanıcıya seçenek olarak sunmak, özellikle tecrübesiz kullanıcılar için kafa karıştırıcı olabilir. Örneğin:

- Kullanıcı, kurulum sırasında **EFI** seçeneğini işaretlediğinde fakat sistem aslında **Legacy BIOS** ise, ya da tam tersi durumda, yanlış seçim yapılabilir.
- Ayrıca, **sda** ve **nvme** diskler arasında farklı kurulum senaryoları gerekebilir.

Bu nedenle hazırlanan kurulum sistemi, aşağıdaki tüm olası senaryolara otomatik cevap verecek şekilde tasarlanmıştır:

- **Legacy BIOS** → **sd*** disk tipi
- **Legacy BIOS** → **nvme*** disk tipi (**desteklenmemektedir**)
- **UEFI** → **sd*** disk tipi
- **UEFI** → **nvme*** disk tipi

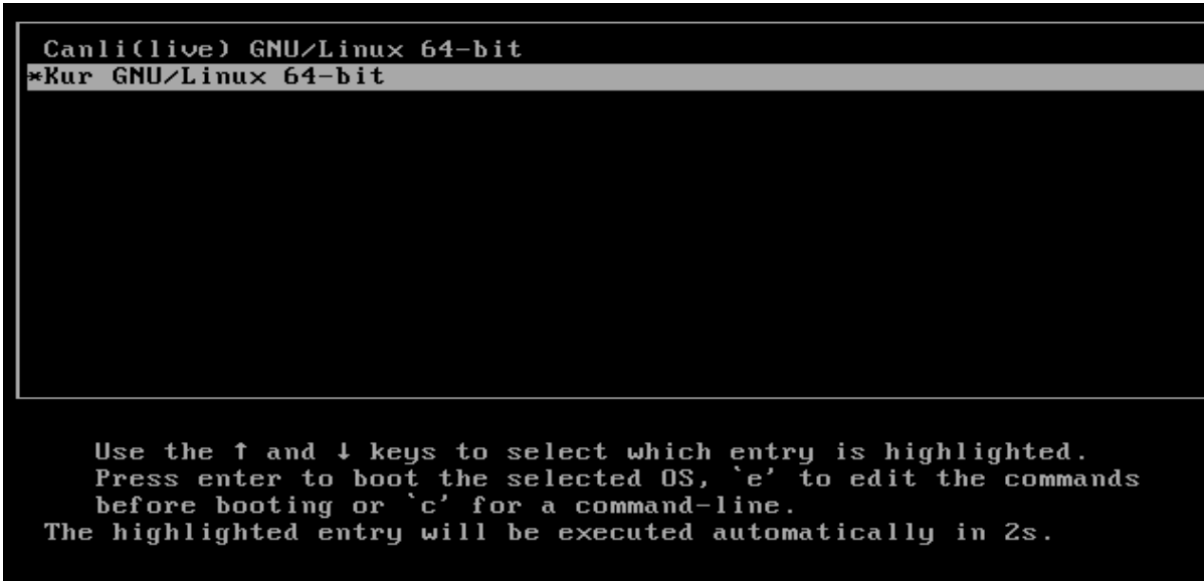
Not

Bu senaryolara göre hazırlanan kurulum scripti, **dialog** aracı kullanılarak oluşturulmuştur. İlgili kurulum scriptleri, **base-file** paketinin içindeki **files.rar** arşivinde yer almaktadır.

Bu bölümde yalnızca **Legacy BIOS** ve **UEFI** sistem kurulumunun genel adımları anlatılacaktır. Bu yöntemleri kendi ihtiyaçlarınıza göre düzenleyerek kullanabilirsiniz.

Sistem Kurulumu

Sistemin kurulumu için resimlerde görünen sıraya göre seçimler yapmalıyız.



Kurulum menüsünde kullanıcı adları ve parolaları, klavye varsayılan olarak;

- Kullanıcı: root Parola: 1
- Kullanıcı: user1 Parola: 1
- Dil : tr_TR
- Klavye : trq

menüden değişiklik yapabilirsiniz. Değişiklik yapmadan sadece kurulum diskini ve disk bölümünü seçip Install(Yükle) işlemi yapabilirsiniz.



```

kurulumu geçildi.....
legacy kurulum .....
Kurulum Yapılacak Disk sda
mount: /kaynak: WARNING: source write-protected, mounted read-only.
*****disk bölümleri biçimlendiriliyor *****
e2fsck 1.47.0 (5-Feb-2023)
e2fsck: need terminal for interactive repairs
tune2fs 1.47.0 (5-Feb-2023)
Recovering journal.

This operation requires a freshly checked filesystem.
Please run e2fsck -f on the filesystem.

mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2096896 4k blocks and 524288 inodes
Filesystem UUID: 9ea082d0-a034-4dab-8e2f-564e68c99c9c
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

Information: You may need to update /etc/fstab.

***** kurulum başladı*****
Sistem yüklenmeye başlandı. Tahmini 3-5 dakika sürecektir.. Lütfen bekleyiniz.....
.....

```

Kaynak:

- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)#UEFI_sistemler](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)#UEFI_sistemler) - 08/07/2025
- https://tldp.org/HOWTO/html_single/SquashFS-HOWTO/ - 09/07/2025
- <https://askubuntu.com/questions/437880/extract-a-squashfs-to-an-existing-directory> - 11/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_2ad2ac6b-d067-40b0-9b55-46db0c2c98dc - 10/07/2025

X Pencere Sistemi

Xorg Çalıştırma

Xorg Yapılandırma Dosyası

Xorg, genellikle /etc/X11/xorg.conf dosyasını kullanır. Eğer bu dosya yoksa, Xorg varsayılan ayarlarla çalışacaktır. Ancak, özel ayarlar yapmak istiyorsanız, bu dosyayı oluşturmanız gerekebilir. Aşağıdaki komut ile bir yapılandırma dosyası oluşturabilirsiniz:

```
sudo X -configure
```

Bu komut, xorg.conf.new adında bir dosya oluşturacaktır. Bu dosyayı /etc/X11/xorg.conf olarak kopyalayabilirsiniz:

```
sudo cp ~/xorg.conf.new /etc/X11/xorg.conf
```

Xwrapper.config

```
echo needs_root_rights=yes>/etc/X11/Xwrapper.config  
echo allowed_users=anybody>>/etc/X11/Xwrapper.config
```

Kullanıcı Ayarı

Kullanıcının tty ve wheel grubunda olması lazım ayrıca **/dev/tty*** dosyalarının grub ve izinleri ayarlanmalıdır. ... code-block:: shell

```
chmod 620 /dev/tty* chgrp tty /dev/tty* usermod -a -G tty by addgroup wheel usermod -a -G wheel by
```

Xorg'u Elle Başlatma

Xorg'u elle başlatmak için terminalde aşağıdaki komutu kullanabilirsiniz:

```
export DISPLAY=:0  
Xorg:0
```

Bu şekilde çalıştırmak yerine;

```
startx
```

Bu komut, varsayılan kullanıcı arayüzünü başlatacaktır. Eğer belirli bir pencere yöneticisi veya masaüstü ortamı ile başlatmak istiyorsanız, ~/.xinitrc dosyasını düzenlemeniz gerekebilir. Örneğin, openbox kullanmak istiyorsanız, dosyanın içeriği şöyle olmalıdır:

```
exec openbox-session
```

Xorg'u Durdurma

Xorg'u durdurmak için, terminalde Ctrl + Alt + Backspace tuş kombinasyonunu kullanabilirsiniz. Bu, X sunucusunu kapatacaktır.

Xorg Hata ayıklama

Hata Ayıklama

Eğer Xorg başlatılamıyorsa, hata ayıklamak için aşağıdaki komutu kullanarak log dosyalarını kontrol edebilirsiniz:

```
cat /var/log/Xorg.0.log
```

Bu dosya, Xorg'un başlatılması sırasında oluşan hataları ve uyarıları içerecektir. Hataları inceleyerek sorunu çözmeye çalışabilirsiniz.

Yardımcı Konular

ssh ile Sanal Makinaya Bağlanma

Bir sanal makineye (**VM**) SSH üzerinden bağlanmak, uzak sistem yönetimi için yaygın ve güvenli bir yöntemdir. Aşağıdaki adımları takip ederek sanal makinana kolayca bağlanabilirsin.

1. SSH Sunucusunun Kurulu ve Çalışır Olmalıdır

Temel Sistem:

```
klykur openssh  
rc-update add sshd  
rc-service sshd start
```

2. Sanal Makina IP Adresi

SanalMakina içerisinde aşağıdaki komutlardan biriyle IP adresini öğrenilir:

```
ip a
```

veya

```
hostname -I
```

Örnek IP adresi:

```
192.168.1.101
```

3. sshd_config Dosyasını Düzenle

/etc/ssh/sshd_config config dosyasını nano ile açıp,

```
nano /etc/ssh/sshd_config
```

#PermitRootLogin prohibit-password satırını

PermitRootLogin yes olarak değiştirilir.

4. SSH ile Bağlantı Kur

Ana sistemden terminal açarak aşağıdaki komut ile SSH bağlantısı yapılır:

```
ssh kullanıcı_adı@ip_adresi
```

Örnek:

```
ssh kly@192.168.1.101
```

İlk bağlantıda şu uyarıyı verebilir:

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

yes yazıp enter'a basılıp, şifreyi girildiğinde **SSH** bağlantısı hazır olur.

sftp ile Sanal Makinaya Bağlanma

SFTP (Secure File Transfer Protocol), SSH protokolü üzerinden güvenli dosya transferi yapmayı sağlar. Bir sanal makineye bağlanarak dosya alışverişi yapmak için aşağıdaki adımları takip edebilirsiniz.

1. SSH Sunucusunun Kurulu ve Çalışır Olmalıdır

SFTP, SSH servisi üzerinden çalışır. Sanal makinede SSH sunucusunun yüklü ve aktif olması gerekir.

```
sudo rc-status sshd
```

Eğer kurulu değilse:

```
klykur openssh
```

2. Sanal Makinanın IP Adresini Öğren

Sanal makine içinde aşağıdaki komutlardan biriyle IP adresini öğrenilir:

```
ip a
```

veya

```
hostname -I
```

Örnek IP adresi:

```
192.168.1.101
```

3. SFTP ile Bağlantı Kur

Ana makineden terminal açarak aşağıdaki komut ile SFTP bağlantısı başlatılır:

```
sftp kullanıcı_adı@ip_adresi
```

Örnek:

```
sftp kly@192.168.1.101
```

Bağlandıktan sonra şu ekranı görürsünüz:

```
sftp>
```

Aşağıdaki komutları kullanabilirsiniz:

- **ls** : Uzak sunucudaki dizin içeriğini listeler.
- **lcd /yerel/yol** : Yerel sistemdeki dizini değiştirir.
- **cd /uzak/yol** : Uzak sunucudaki dizini değiştirir.
- **get dosya.txt** : Uzak dosyayı indirir.
- **put dosya.txt** : Yerel dosyayı sunucuya yükler.

- **exit** : Bağlantıyı sonlandırır.

4. Grafik Arayüz SFTP Alternatifleri

Linux ortamlarında, dosya yöneticileri ve FTP programları ile de SFTP bağlantısı kurulabilir.

GNOME Nautilus / Cinnamon Nemo ile SFTP bağlantısı için:

Adres çubuğuna bağlantı bilgisi şu formatta yazılır:

```
sftp://kly@192.168.1.101
```

FileZilla gibi bir ftp programı ile aşağıdaki ayarlar kullanılarak SFTP bağlantısı yapılır:

- Host: *sftp://192.168.1.101*
- Username: *kly*
- Password: şifre
- Port: 22

Not: Sanal makinenizin ağ ayarlarının (NAT veya Bridged) ve güvenlik duvarı yapılandırmasının doğru olması gerekir.

scp ile Sanal Makinaya Dosya ve Dizin Kopyalama

SCP (Secure Copy Protocol), SSH üzerinden güvenli şekilde dosya ve dizin kopyalamak için kullanılan bir komut satırı aracıdır.

Sanal makinaya dosya veya dizin göndermek için aşağıdaki yöntemler kullanılabilir.

1. Dosya Kopyalama

Yerel makinedeki bir dosyayı sanal makinaya kopyalamak için:

```
scp /yerel/yol/dosya.txt kullanıcı_adı@ip_adresi:/uzak/yol/
```

Örnek:

```
scp ~/Belgeler/rapor.pdf kly@192.168.1.101:/home/kly/
```

2. Dizin Kopyalama

Bir dizini ve içeriğini kopyalamak için `-r` (recursive) seçeneği kullanılır:

```
scp -r /yerel/yol/dizin kullanıcı_adı@ip_adresi:/uzak/yol/
```

Örnek:

```
scp -r ~/Projeler/myproject kly@192.168.1.101:/home/kly/
```

3. Sanal Makinadan Dosya veya Dizin Kopyalama

Ters yönde, sanal makinadan yerel makineye dosya veya dizin kopyalamak için de benzer komut kullanılır:

```
scp kullanıcı_adı@ip_adresi:/uzak/yol/dosya.txt /yerel/yol/
```

```
scp kly@192.168.1.101:/home/kly/rapor.pdf ~/Belgeler/
```

veya dizin için:

```
scp -r kullanıcı_adı@ip_adresi:/uzak/yol/dizin /yerel/yol/
```

```
scp -r kly@192.168.1.101:/home/kly/ ~/Projeler/myproject
```

4. Önemli Notlar

- Sanal makinenin IP adresi ve kullanıcı adı doğru olmalıdır.
- SSH servisi sanal makinada çalışıyor olmalıdır.
- Güvenlik duvarı ve ağ ayarları dosya transferine izin vermelidir.
- Kopyalama sırasında parola istenebilir veya SSH anahtarları kullanılabilir.

apt Paket Sistemi

apt paket sistemi debian tabanlı sistemlerde paket yönetimi için kullanılan bir uygulamadır. Temel işlemler şunlardır.

Yerel Paket İndexini Güncelleme

```
sudo apt update
```

Paket Yükleme

```
sudo apt install paket_adi
```

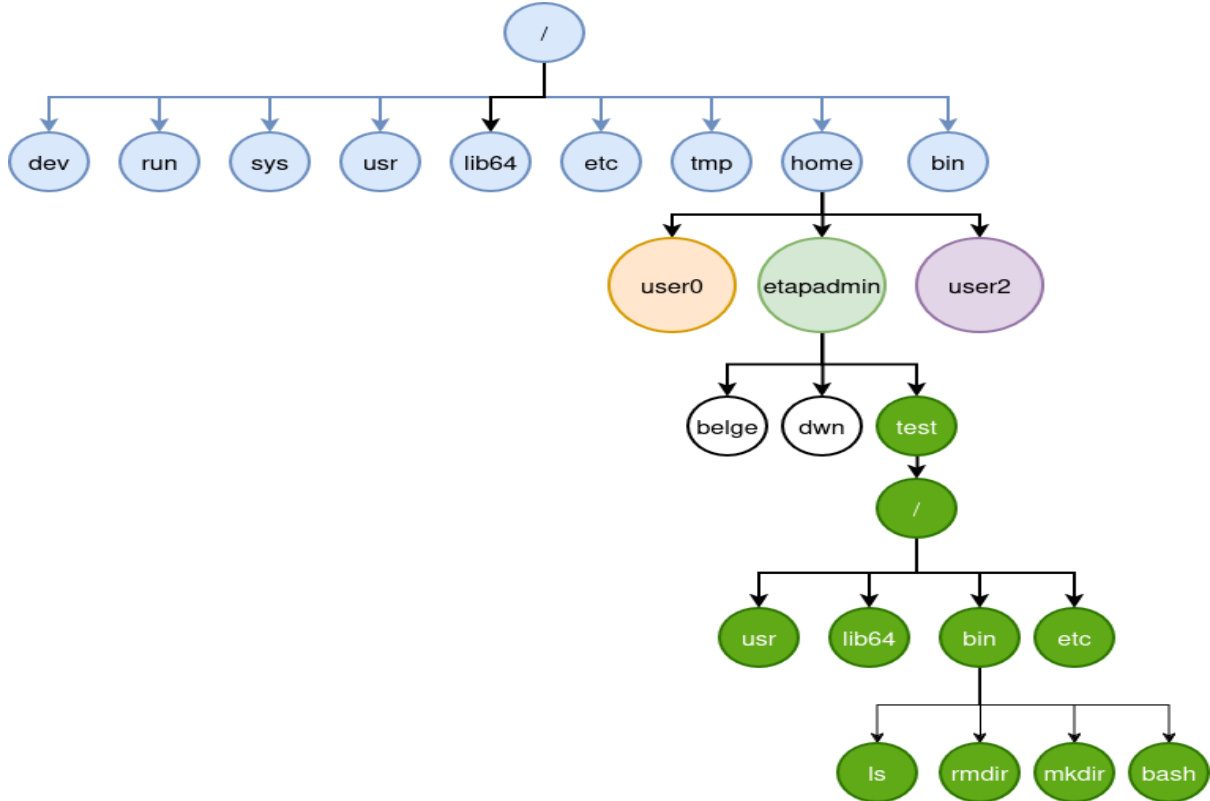
Paket Silme

```
sudo apt remove paket_adi
```

Chroot Nedir?

chroot komutu çalışan sistem üzerinde belirli bir klasöre root yetkisi verip sadece o klasörü sanki linux sistemi gibi çalıştıran bir komuttur. Sağladığı avantajlar çok fazladır. Bunlar;

- Sistem tasarlama
- Sistem üzerinde yeni dağıtımlara müdahale etme ve sorun çözme
- Kullanıcı kendine özel geliştirme ortamı oluşturabilir.
- Yazılım bağımlıkları sorunlarına çözüm olabilir.
- Kullanıcıya sadece kendisine verilen alanda sınırsız yetki verme vb.



Yukarıdaki resimde user1 altında wrk dizini altına yeni bir sistem kurulmuş gibi yapılandırmayı gerçekleştirmiş.

/home/etapadmin/test dizinindeki sistem üzerinde sisteme erişmek için;

```
# sisteme erişim yapıldı.  
sudo chroot /home/etapadmin/test
```

/home/etapadmin/test dizinindeki sistem üzerinde sistemi silmek için;

```
# sistem silindi  
sudo rm -rf /home/etapadmin/test
```


Yeni sistem tasarlamak ve erişmek için temel komutları ve komut yorumlayıcısının olması gerekmektedir. Bunun için bize gerekli olan komutları bu yapının içine koymamız gerekmektedir. Örneğin ls komutu için doğrudan çalışıp çalışmadığını ldd komutu ile kontrol edelim.

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ ldd /bin/ls  
linux-vdso.so.1 (0x00007ffc68471000)  
libgtk3-nocsd.so.0 => /usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0 (0x00007ff3fa9db000)  
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007ff3fa9ad000)  
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ff3fa7cc000)  
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007ff3fa7c7000)  
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007ff3fa7c2000)  
libpcre2-8.so.0 => /usr/lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007ff3fa726000)  
/lib64/ld-linux-x86-64.so.2 (0x00007ff3faa2a000)  
etapadmin@etahta:~$
```

Görüldüğü gibi ls komutunun çalışması için bağımlı olduğu kütüphane dosyaları bulunmaktadır. Bağımlı olduğu dosyaları yeni oluşturduğumuz sistem dizinine aynı dizin yapısında kopyalamamız gerekmektedir. Bu dosyalar eksiksiz olursa ls komutu çalışacaktır. Fakat bu işlemi tek tek yapmamız çok zahmetli bir işlemdir. Bu işi yapacak script dosyası aşağıda verilmiştir.

Bağımlılık Scripti

lddscript.sh

```
#!/bin/bash  
  
if [ ${#} != 2 ]  
then  
    echo "usage $0 PATH_TO_BINARY target_folder"  
    exit 1  
fi  
path_to_binary="$1"  
target_folder="$2"  
  
# if we cannot find the the binary we have to abort  
if [ ! -f "${path_to_binary}" ]  
then  
    echo "The file '${path_to_binary}' was not found. Aborting!"  
    exit 1  
fi  
  
echo "---> copy binary itself" # copy the binary itself  
cp --parents -v "${path_to_binary}" "${target_folder}"  
  
echo "---> copy libraries" # copy the library dependencies  
ldd "${path_to_binary}" | awk -F'[> ]' '{print $(NF-1)}' | while read -r lib  
do  
    [ -f "$lib" ] && cp -v --parents "$lib" "${target_folder}"  
done
```

Basit Sistem Oluşturma

Bu örnekte kullanıcının(etapadmin) ev dizinine(/home/etapadmin) test dizini oluşturuldu ve işlemler yapıldı. ls, rmdir, mkdir ve bash komutlarından oluşan sistem hazırlama.

Sistem Dizinin Oluşturulması

```
# ev dizinine test dizini oluşturuldu.  
mkdir /home/etapadmin/test/
```

/home/etapadmin/ dizinine **Bağımlılık Scripti** kodunu **lddscripts.sh** oluşturalım.

ls Komutu

```
# ls komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/ls /home/etapadmin/test/
```



```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/ls /home/etapadmin/test/  
---> copy binary itself  
'/bin -> /home/etapadmin/test/bin  
'/bin/ls' -> '/home/etapadmin/test/bin/ls'  
---> copy libraries  
'usr -> /home/etapadmin/test/usr  
'usr/lib -> /home/etapadmin/test/usr/lib  
'usr/lib/x86_64-linux-gnu -> /home/etapadmin/test/usr/lib/x86_64-linux-gnu  
'usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'lib -> /home/etapadmin/test/lib  
'lib/x86_64-linux-gnu -> /home/etapadmin/test/lib/x86_64-linux-gnu  
'lib/x86_64-linux-gnu/libselinux.so.1' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libselinux.so.1'  
'lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'usr/lib/x86_64-linux-gnu/libpcre2-8.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0'  
'lib64 -> /home/etapadmin/test/lib64  
'lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

Bu işlemi diğer komutlar içinde sırasıyla yapmamız gerekmektedir.

rmdir Komutu

```
# rmdir komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/rmdir /home/etapadmin/test/
```



```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/rmdir /home/etapadmin/test/  
---> copy binary itself  
'/bin/rmdir' -> '/home/etapadmin/test/bin/rmdir'  
---> copy libraries  
'usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

mkdir Komutu

```
# mkdir komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/mkdir /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/mkdir /home/etapadmin/test/  
---> copy binary itself  
'/bin/mkdir' -> '/home/etapadmin/test/bin/mkdir'  
---> copy libraries  
'/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'/lib/x86_64-linux-gnu/libselinux.so.1' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libselinux.so.1'  
'/lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'/lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'/lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0'  
'/lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

bash Komutu

```
# bash komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/bash /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/bash /home/etapadmin/test/  
---> copy binary itself  
'/bin/bash' -> '/home/etapadmin/test/bin/bash'  
---> copy libraries  
'/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'/lib/x86_64-linux-gnu/libtinfo.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libtinfo.so.6'  
'/lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'/lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'/lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'/lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

chroot Sistemde Çalışma

```
sudo chroot /home/etapadmin/test komutunu kullanmalıyız.
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ sudo chroot /home/etapadmin/test  
[sudo] password for etapadmin:  
bash-5.2# ls  
bin lib lib64 usr  
bash-5.2# mkdir abc  
bash-5.2# ls  
abc bin lib lib64 usr  
bash-5.2# rmdir abc  
bash-5.2# ls  
bin lib lib64 usr  
bash-5.2# pwd  
/  
bash-5.2# ldd  
bash: ldd: command not found  
bash-5.2# exit  
exit  
etapadmin@etahta:~$
```

- **abc** dizini oluşturuldu, **abc** dizini silindi, **pwd** komutuyla konum öğrenildi, **ldd** komutu sistemimizde olmadığından hata verdi.
- Çıkış için ise **exit** komutu kullanılarak sistemden çıkıldı.

Kaynak:

<https://stackoverflow.com/questions/64838052/how-to-delete-n-characters-appended-to-ldd-list>
<https://app.diagrams.net/>

Kaynak Kod Derleme

Bir uygulamanın kodları genellikle çalışmaz(python benzeri kodlar istisna). Bu kodlardan sistemlerin çalışması için çalışabilir dosyalar üretilir(linuxta ikili dosya, elf, windowsta exe, com vb.). Bu çalışabilir dosyaları koddan oluştururken iki farklı şekilde oluşturabiliriz.

1- **Paylaşımlı Derleme(dynamic)**: Kendine lazım olan kütüphaneleri sistem üzerindeki başka uygulamalarla ortak kullanır. 2- **Paylaşımsız, gömülü(static)**: Kendisine lazım olan kütüphaneleri kendi içinde barındırır(portable uygulama gibi).

Şimdi aşağıdaki kaynak kodumuzu iki farklı yöntemle derleyelim.

```
//main.c dosyamız
#include <stdio.h>
void main(){
    printf("Merhaba\n");
}
```

2-Paylaşımlı Derleme(dynamic):

Derlenen uygulama sistemde bulunan kütüphaneleri kullanacak şekilde derlenmesidir. Uygulama boyutu küçüktür, taşınabilirliği sınırlanabilir. Aşağıdaki gibi derlenir.

```
gcc -o main main.c
```

main.c kodumuzu **main** adında ikili çalışabilir dosyaya dönüştürdük. **ldd** komutuyla **main** dosyasının kullandığı kütüphaneler öğrenilir.

```
unset LD_PRELOAD
ldd ./main
    linux-vdso.so.1 (0x00007ffdb3bb9000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2c53fe0000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f2c541e7000)
```

Burada **libc.so.6** ve **ld-linux-x86_64.so.2** dosyaları **glibc** tarafından sağlanır. Derlenmiş dosyanın çalışması için tüm bağımlılıklarının sistemde bulunması gereklidir. Sadece gerekli olan kütüphaneleri görmek için **readelf -d** komutu kullanılabilir. Aşağıda gerekli olan kütüphaneler listeleniyor.

```
readelf -d ./main | grep -i needed
0x0000000000000001 (NEEDED)                               Paylaşımlı kitaplık: [libc.so.6]
```

ldconfig

Sistemdeki kütüphanelerin konumlarını **/etc/ld.so.conf** dosyasına bakarak belirler.

```
#/etc/ld.so.conf dosyası
/usr/lib64
/usr/lib
/lib64
/lib
```

Kütüphanelerde değişiklik yapılmışsa ve hemen bu değişikliği sistemin görmesini istersek **ldconfig** komutu kullanılmalıdır.

2-Paylaşımsız, gömülü(static):

Derlenen uygulama sistemde bulunan ve çalışması için gerekli olan kütüphaneleri uygulama içine dahil eden bir derleme yöntemidir. Uygulamamızı static derlemek için **-static** parametresi ekleyerek derlenir.

```
gcc -o main main.c -static
```

Paylaşımlı(dynamic) derleme işleminde bağımlı olduğu dosyaları **ldd** komutunu kullanarak öğrenmiştik. Şimdi paylaşımsız derlediğimiz **main** dosyasında bağımlı olduğu kütüphaneleri kontrol ediyoruz.

```
ldd main  
not a dynamic executable
```

Bağımlı kütüphaneler yerine **not a dynamic executable** mesajı gördük. Bunun anlamı çalışması için hiçbir kütüphaneye ihtiyaç duymaz. Bu bir avantaj ve taşınabilirliği artırır. Deventajı ise boyutu büyük olur. İhtiyaca göre paylaşımlı veya paylaşımsız derleme tercih edilir.

Derleme Araçları

Linux sistemlerinde kodlarımızı derlemek kullanılan uygulamalara derleme araçları denir. Birden fazla araç olsada en temel derleme araçları **make**, **cmake**, **meson**, **python**.

1-make

make, yazılım geliştirme süreçlerinde sıkça kullanılan bir araçtır. Özellikle C ve C++ gibi dillerde projelerin derlenmesi ve yönetilmesi için kullanılır.

Aşağıdaki C kodumuzu(merhaba.c dosyası) make ile nasıl derleyeceğimizi anlatalım;

```
#include <stdio.h>
int main(){
    puts("Merhaba");
    return 0;
}
```

Makefile Nedir?

Makefile, make aracının nasıl çalışacağını belirten bir dosyadır. İçinde hedefler, bağımlılıklar ve komutlar bulunur. Örneğin, bir C programını derlemek için aşağıdaki gibi basit bir Makefile oluşturabilirsiniz. Makefile ve merhaba.c dosyası aynı konumda olmalıdır.

```
CC=gcc
CFLAGS=-I.
all: program
program: merhaba.o
    $(CC) -o program merhaba.o
merhaba.o: merhaba.c
    $(CC) -c merhaba.c $(CFLAGS)
clean:
    rm -f *.o program
```

merhaba dosyasından **program** adında bir ikili dosya oluşturmak için aşağıdaki komutlar Makefile ve merhaba.c dosyasının olduğu konumda çalıştırılır.

```
make
make install
```

Genellikle Makefile dosyası olmaz. Onun yerine **configure.ac** dosyası olur. **configure.ac** dosyasından **Makefile** dosyası elde etmek için, öncelikle **autoconf** ve **automake** araçlarının sisteminizde kurulu olması gerekmektedir.

autoreconf -fvi komutu çalıştırılarak configure dosyamızı üretir. Bu araçlar, yapılandırma dosyalarınızı işleyerek gerekli **Makefile** dosyalarını oluşturmanıza yardımcı olur.

configure komutunun devamında **--prefix** dışında başka parametreleride olabilir. Bu parametreleri "Read.me" dosyası içerisinde bulunabilir veya **configure --help** komutu kaynak kodların olduğu konumda kullanılarak görülebilir. Bu kaynak kod aşağıdaki gibi derlenir.

```
$ autoreconf -fvi
$ ./configure --prefix=/usr
$ make
$ make install
```

2-cmake

CMake, projelerin derlenmesi ve yapılandırılması için kullanılan bir araçtır. Bir paketi CMake ile derlemek için genellikle aşağıdaki adımları izleriz:

İlk olarak, projenin kök dizininde bir CMakeLists.txt dosyası oluşturun. Ardından, CMake komutunu kullanarak projeyi yapılandırın ve derleyin. Örneğin:

```
mkdir compile_packages
cd compile_packages
cmake ..
make
```

Bu adımları takip ederek, CMake ile paketinizi başarıyla derleyebilirsiniz.

CMake'in esnekliği ve kullanım kolaylığı sayesinde paketlerin derlenmesi ve yapılandırılması oldukça kolay hale gelir.

Bu kaynak kod aşağıdaki gibi derlenir:

```
mkdir compile_packages
cd compile_packages
cmake ..
make
make install
```

3-meson

Meson, modern bir yapılandırma betik dili ve Ninja ise hızlı bir derleyici araçtır. Bir paketi derlemek için öncelikle Meson ile yapılandırma dosyalarını oluşturmanız gerekir. Ardından, Ninja derleyici aracını kullanarak bu yapılandırmayı derleyebilirsiniz.

İlk olarak, projenizin dizinine gidin ve Meson ile yapılandırma dosyalarını oluşturun:

```
meson setup builddir --prefix=/usr
```

Sonra, oluşturulan builddir dizinine geçin ve Ninja ile derlemeyi başlatın:

```
ninja -C builddir
```

Bu adımları takip ederek Meson ve Ninja kullanarak paketinizi başarılı bir şekilde derleyebilirsiniz.

Bu kaynak kod aşağıdaki gibi derlenir:

```
# paketin yükleneceği konum
INSTALL_ROOT=/
meson setup builddir --prefix=/usr
ninja -C builddir
INSTALL_ROOT=$INSTALL_ROOT ninja -C builddir install
```

4-python

Python ile paket derlemek için genellikle **setuptools** ve **distutils** gibi kütüphaneler kullanılır. Öncelikle, projenizin kök dizininde bir **setup.py** dosyası oluşturmanız gerekir. Bu dosya, paketin yapılandırmasını ve gereksinimlerini tanımlar.

Örnek bir setup.py dosyası aşağıdaki gibi olabilir:

```
from setuptools import setup

setup(
    name='paket_adi',
    version='1.0',
    packages=['paket'],
    install_requires=[
        'bağımlılık_paketi',
    ],
)
```

Ardından, terminalde projenizin bulunduğu dizine giderek aşağıdaki komutu çalıştırarak paketi derleyebilirsiniz:

Bu komut, paketi dist klasörüne derleyecektir. Artık paketiniz hazır ve dağıtımına uygun hale gelmiştir.

BusyBox

BusyBox, <https://busybox.net/about.html> adresteki bilgilere göre birçok temel Unix aracını tek bir ikili dosya içinde sunan, küçük ve esnek bir programdır. Çoğunlukla **initramfs** sistemlerinde ve gömülü Linux dağıtımlarında tercih edilir. BusyBox ile komutlar şu şekilde çalıştırılabilir:

```
busybox [komut]
```

Bir komutu doğrudan çalıştırabilmek için BusyBox'ı o komut adıyla sembolik bağlamak mümkündür. Örneğin, **tar** komutunu BusyBox üzerinden çalıştırmak için:

```
ln -s /bin/busybox ./tar
./tar
```

Burada busybox içindeki gömülü olan tar kullanmayı gördük. Aslında aklımıza gelen her komutun busybox içinde gömülü hali vardır. Ama bu tüm komutlar için ve tüm parametreleri için geçerli değildir. Busybox içindeki tüm komutların kısayolunu(sembolik bağı) eklemek için aşağıdaki komut kullanılır.

```
busybox --install -s /bin
```

Derleme

Gömülü sistem tasarımı veya **initramfs** içinde kullanılacak busybox **paylaşımsız(static)** derlenir. Bu şekilde derlendiğinde glibc kütüphanelerine ihtiyaç duymadan çalışacaktır.

Şimdi busybox derlemek için <https://busybox.net/> adresinden **stable** başlığı altındaki kaynak kodlarını indiriniz.

```
# kaynak kod indirilir
wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2

# indirilen dosya açılır
tar -xvjf busybox-1.36.1.tar.bz2

# açılan kaynak dosyalarının bulunduğu dizine geçilir
cd busybox-1.36.1/
```

Şimdi kodlarımızı indirdik. derlemek için aşağıdaki komutları çalıştıralım.

```
# varsayılan yapılandırmayı uygula
make defconfig

# static derleme yapacaksak sed ile .config dosyasını düzenliyoruz.
sed -i "s|.*CONFIG_STATIC_LIBGCC .*|CONFIG_STATIC_LIBGCC=y|" .config
sed -i "s|.*CONFIG_STATIC .*|CONFIG_STATIC=y|" .config

# derliyoruz
make

# Static derleme sonucu aşağıdaki gibi görünür.
ldd /bin/busybox
özdevimli bir çalıştırılabilir değil
```

Derleme tamamlandığında, oluşturulan **busybox** ikili dosyası kaynak dizininde bulunur.

OpenRC

Openrc sistem açılışında çalışacak uygulamaları çalıştıran servis yöneticisidir.

Çalıştırılması

Openrc servis yönetiminin çalışması için boot parametrelerine yazılması gerekmektedir. **/boot/grub.cfg** içindeki **linux /vmlinuz init=/usr/sbin/openrc-init root=/dev/sdax** olan satırda **init=/usr/sbin/openrc-init** yazılması gerekmektedir. Artık sistem openrc servis yöneticisi tarafından uygulamalar çalıştırılacak ve sistem hazır hale getirilecek.

openrc Kullanımı

Servis etkinleştirip devre dışı hale getirmek için **rc-update** komutu kullanılır. Aşağıda **udhcpc** internet servisi örnek olarak gösterilmiştir. **/etc/init.d/** konumunda **udhcpc** dosyamızın olması gerekmektedir.

```
# servis etkinleştirmek için
rc-update add udhcpc boot
# servisi devre dışı yapmak için
rc-update del udhcpc boot
# Burada udhcpc servis adı, boot ise runlevel adıdır.
```

Elle **/etc/runlevels/** altında bulunan **boot, default, nonetwork, shutdown, sysinit** dizinlerine **/etc/init.d/** dizininin altındaki dosyaları **In** komutuyla kısayol yaparsak **rc-update add** komutunun yaptığı görevi yapmış oluruz. Kısayolu silerek **rc-update del** komutunun görevini yapmış oluruz.

/etc/runlevels/ altında bulunan **boot, default, nonetwork, shutdown, sysinit** dizinler servis dosyalarımızın hangi sırayla çalışacağını belirleyen dizinlerdir. Mesela **boot** dizini ilk açılış sırasında çalışacak olan servis dosyalarının konulacağı dizindir.

Servisleri başlatıp durdurmak için ise **rc-service** komutu kullanılır.

```
rc-service udhcpc start
# veya şu şekilde de çalıştırılabilir.
/etc/init.d/udhcpc start
```

Servislerin durumunu öğrenmek için **rc-status** komutu kullanılır. Ayrıca sistemdeki servislerin sonraki açılışta hangisinin başlatılacağını öğrenmek için parametresiz olarak **rc-update** kullanabilirsiniz.

```
# şu an hangi servislerin çalıştığını gösterir
rc-status
# sonraki açılışta hangi servislerin çalışacağını gösterir
rc-update
```

Sistemi kapatmak veya yeniden başlatmak için **openrc-shutdown** komutunu kullanabilirsiniz.

```
openrc-shutdown -p 0 # kapatmak için
openrc-shutdown -r 0 # yeniden başlatmak için
```

Servis Dosyası

Openrc servis dosyaları basit birer **bash** betiğidir. Bu betikler **openrc-run** komutu ile çalıştırılır ve çeşitli fonksiyonlardan oluşabilir. Servis dosyaları **/etc/init.d** içerisinde bulunur. Servisleri ayarlamak için ise **/etc/conf.d** içerisine aynı isimle ayar dosyası oluşturabiliriz.

Çalıştırılacak komut, komut parametreleri ve **pidfile** dosyamızı aşağıdaki gibi belirtebiliriz.

```
description="Ornek servis"
command=/usr/bin/ornek-servis
command_args=- -parametre
pidfile=/run/ornek-servis.pid
```

Bununla birlikte **start**, **stop**, **status**, **reload**, **start_pre**, **stop_pre** gibi fonksiyonlar da yazabiliriz.

```
start(){
    ebegin "Starting ${RC_SVCNAME}"
    start-stop-daemon --start --pidfile "/run/servis.pid" --exec /usr/bin/ornek-servis --parametre
}
```

Servis bağımlılıklarını belirtmek için ise **depend** fonksiyonu kullanılır.

```
depend() {
    need localmount
    after dbus
}
```

Qemu Kullanımı

qemu açık kaynaklı Virtual Box, Vmware benzeri sanallaştırma aracıdır.

Sisteme Kurulum

```
sudo apt update
sudo apt install qemu-system-x86 qemu-utils
```

qemu Kullanımı

```
# 30GB disk oluşturuldu.
qemu-img create disk.img 30G
qemu-system-x86_64 --enable-kvm -hda disk.img -m 2G -cdrom etahta.iso
# qemu-system-x86_64 --enable-kvm -hda disk.img -m 2G #sadece disk ile çalıştırılıyor
# qemu-system-x86_64 -m 2G -cdrom etahta.iso #sadece iso dosyası ile çalıştırma
```

Sistem Hızlandırılması

--enable-kvm eğer sistem disk ile çalıştırıldığında bu parametre eklenmezse yavaş çalışacaktır.

Boot Menu Açma

Sistemin diskten mi imajdan mı başlayacağını başlangıçta belirlemek için boot menu gelmesini istersek aşağıdaki gibi komut satırına seçenek eklemeliyiz.

```
qemu-system-x86_64 --enable-kvm -cdrom distro.iso -hda disk.img -m 4G -boot menu=on
```

Uefi kurulum için:

```
sudo apt-get install ovmf
```

```
qemu-system-x86_64 --enable-kvm -bios /usr/share/ovmf/OVMF.fd -cdrom distro.iso -hda disk.img -m 4G -boot menu=on
```

qemu Host Erişimi:

İstemci bilgisayar ip'si:10.0.2.15 ve ana bilgisayar 10.0.0.2 olarak ayarlıyord.

vmlinuz ve initrd

qemu ile vmlinuz ve initrd.img dosyaları iso olmadan test edilebilir.

```
qemu-system-x86_64 --enable-kvm -kernel /boot/vmlinuz-5.17 -initrd /home/deneme/initrd.img -append "quiet" -m 512m
```

qemu Terminal Yönlendirmesi

```
qemu-system-x86_64 --enable-kvm -kernel vmlinuz -initrd initrd.img -m 3G -serial stdio -append "console=ttyS0"
```

Diskteki Sistemin Açılışını Terminale Yönlendirme

```
qemu-system-x86_64 -nographic -kernel boot/vmlinuz -hda disk.img -append console=ttyS0
```

Kaynak: | <https://www.ubuntubuzz.com/2021/04/how-to-boot-uefi-on-qemu.html>

Live Sistem Oluşturma

Canlı sistem oluşturma veya RAM üzerinden çalışan sistem hazırlamak için SquashFS dosya sisteminde dağıtım sıkıştırılmalıdır. SquashFS, Linux işletim sistemlerinde sıkıştırılmış bir dosya sistemidir. Sistemimizi sıkıştırır ve ardından salt okunur bir dosya sistemine dönüştürür.

SquashFS Oluşturma

```
# mksquashfs input_source output/filesystem.squashfs -comp xz -wildcards
mksquashfs initrd $HOME/distro/filesystem.squashfs -comp xz -wildcards
```

Cdrom Erişimi

Cd veya Dvd aygıtı Linux sistemlerinde /dev/sr0 aygıt dosyası olarak erişilir. Cd içeriği üzerinde okuma yapmak için aşağıdaki komutu kullanabiliriz.

```
cat /dev/sr0
```

Cdrom Bağlama

```
mkdir cdrom
mount /dev/sr0 /cdrom
```

Bu işlem sonucunda cdrom bağlanmış olacaktır. ISO dosyamızın içerisine erişebiliriz.

Squashfs Dosyasını Bulma

Genellikle isoların içine squashfs dosyası oluşturulur. Bu sayede live yükleme yapılabilir. Örneğin, /live/filesystem.squashfs imaj dosyalarında konumudur.

Squashfs Bağlama

Squashfs dosyasını bağlamadan önce loop modülünün yüklü olması gerekmektedir. Eğer yüklemeyerseniz;

```
# loop modülü yüklenir.
modprobe loop
mkdir canli
mount -t squashfs -o loop cdrom/live/filesystem.squashfs /canli
```

Squashfs Sistemine Geçiş

Yukarıdaki adımlarda squashfs dosyamızı /canli adında dizine bağlamış olduk. Bu aşamadan sonra sistemimizin bir kopyası olan squashfs canlıdan erişilebilir veya sistemi buradan başlatabiliriz.

Squashfs dosya sistemimize bağlanmak için;

```
chroot canli /bin/bash
```

Bu işlemin yerine exec komutuyla bağlanırsak sistemimiz id "1" değeriyle çalıştıracaktır. Eğer sistemin bu dosya sistemiyle açılmasını istiyorsak exec ile çalıştırıp id=1 olmasına dikkat etmeliyiz.

kmod Nedir?

Linux çekirdeği ile donanım arasındaki haberleşmeyi sağlayan kod parçalarıdır. Bu kod parçalarını kernele eklediğimizde kerneli tekrardan derlememiz gerekmektedir. Her eklenen koddan sonra kernel derleme, kod çıkarttığımızda kernel derlemek ciddi bir iş yükü ve karmaşa yaratacaktır.

Bu sorunların çözümü için modul vardır. moduller kernele istediğimiz kod parçalarını ekleme ya da çıkartma yapabiliyoruz. Bu işlemleri yaparken kernel derleme işlemi yapmamıza gerek yok.

Kernele modul yükleme kaldırma için kmod aracı kullanılmaktadır. kmod aracı;

```
ln -s kmod /bin/depmod
ln -s kmod /bin/insmod
ln -s kmod /initrd/bin/lsmmod
ln -s kmod /bin/modinfo
ln -s kmod /bin/modprobe
ln -s kmod /bin/rmmod
```

şeklinde sembolik bağlarla yeni araçlar oluşturulmuştur.

lsmod : yüklü modulleri listeler

insmod: tek bir modul yükler

rmmod: tek bir modul siler

modinfo: modul hakkında bilgi alınır

modprobe: insmod komutunun aynısı fakat daha işlevseldir. module ait bağımlı olduğu modülleride yüklemektedir. modprobe modülü /lib/modules/ dizini altında aramaktadır.

depmod: /lib/modules dizinindeki modüllerin listesini günceller. Fakat başka bir dizinde ise basedir=konum şeklinde belirtmek gerekir. konum dizininde /lib/modules/** şeklinde kalsörler olmalıdır.

strip

strip komutu, derlenmiş program veya paylaşılan kütüphanelerin dosya boyutunu küçültmek için kullanılır. Derleme sırasında eklenen gereksiz sembol ve hata ayıklama bilgilerini kaldırarak dosyayı küçültür.

strip komutunu kullanmak için terminalde şu komutu yazabilirsiniz:

```
strip dosya_adı
```

Burada "dosya_adı", boyutu azaltılacak dosyanın adıdır. Örneğin:

```
strip program
```

Bu komut, özellikle Linux ve Unix sistemlerinde yaygın kullanılır. Dosya boyutunu küçültürken disk alanı tasarrufu sağlar ve programların dağıtımı için uygundur.

Ancak, hata ayıklama veya sembol bilgilerine ihtiyaç duyduğunuz durumlarda strip komutunu kullanmamanız gerekir.

Özetle, strip komutu derlenmiş dosyaların gereksiz bilgilerini temizleyerek boyutlarını azaltan basit ve etkili bir Linux aracıdır.

Kullanıcı Ekleme

linux sistemlerine kullanıcı eklemek için iki farklı komut bulunur. Bu komutlar adduser ve useradd komutlarıdır.

adduser:

Kullanıcı dostu bir arayüz sunar. Birçok aşamayı bizim adımıza yapar. Temelde useradd komutunu kullanarak yazılan bir script olarak düşünebiliriz.

```
# adduser komutuyla kullanıcı oluşturma
sudo adduser yeni_kullanici
```

useradd:

Kullanıcıdan tüm parametreleri girmesini ister. Bu sebepten çok tercih edilmemektedir. Fakat özel bir şekilde kullanıcı oluşturulmak istenildiğinde tercih edilir. Bu dokümanda kurulum aşamalarında useradd komutu kullanıldı.

```
# useradd komutuyla kullanıcı oluşturma
sudo useradd -m -s /bin/bash yeni_kullanici -d /home/yeni_kullanici
# -s/bin/bash : kullanıcının kullanacağı shell belirtiliyor.
# -d /home/yeni_kullanici : home dizinine oluşturulacak kullanıcı dizini belirtiyoruz
```


github

GitHub üzerinde yeni bir depo açmak oldukça basit bir işlemdir. Aşağıdaki adımları takip ederek hızlıca kendi deponuzu oluşturabilirsiniz:

- **GitHub Hesabınıza Giriş Yapın:** GitHub ana sayfasına gidin ve hesabınıza giriş yapın. Eğer bir hesabınız yoksa, öncelikle bir hesap oluşturmalısınız.
- **Yeni Depo Oluşturma:** Sağ üst köşede bulunan "+" simgesine tıklayın ve "New repository" seçeneğini seçin.
- **Depo Bilgilerini Girin:** Açılan sayfada, depo adını (repository name) ve isteğe bağlı olarak bir açıklama (description) girin. Depo özel (private) veya herkese açık (public) olarak ayarlanabilir.
- **İlk Dosyayı Oluşturma:** "Initialize this repository with a README" seçeneğini işaretleyerek, depo oluşturulduğunda otomatik olarak bir README dosyası oluşturabilirsiniz.
- **Depoyu Oluşturma:** "Create repository" butonuna tıklayarak deponuzu oluşturun.

Artık GitHub üzerinde yeni bir deponuz var! Bu depoyu yerel bilgisayarınıza klonlayarak projelerinizi yönetmeye başlayabilirsiniz. Örneğin, terminalde şu komutu kullanarak deponuzu klonlayabilirsiniz:

```
git clone https://github.com/kullaniciadi/depoadi.git
```

Bu adımları takip ederek GitHub üzerinde kolayca depo açabilirsiniz.

X11 Kullanıcısının Tespiti

Linux ortamında masaüstüdeyken sudo ile script çalıştırıldığında masaüstünü açtığımız kullanıcının tespiti aşağıdaki kodla yapılabilir.

```
#!/bin/bash

# Detect the name of the display in use
display=":$(ls /tmp/.X11-unix/* | sed 's#/tmp/.X11-unix/X##' | head -n 1)"

# Detect the user using such display
user=$(who | grep '('$display')' | awk '{print $1}')

# Detect the id of the user
uid=$(id -u $user)

echo $user $uid
```

Kaynaklar

- <https://tr.wikipedia.org/wiki/Linux> - 02/07/2025
- <https://www.subrat.info/build-kernel-and-userspace/> - 08/07/2025
- <https://medium.com/@chienhaotan/compiling-and-running-a-minimal-kernel-with-busybox-bfc45a991017> - 08/07/2025
- <https://stackoverflow.com/questions/64838052/how-to-delete-n-characters-appended-to-ldd-list> - 20/06/2025
- <https://gist.github.com/bluedragon1221/a58b0e1ed4492b44aa530f4db0ffef85> - 09/07/2025
- <https://app.diagrams.net/>
- <https://www.ubuntubuzz.com/2021/04/how-to-boot-uefi-on-qemu.html> - 20/06/2024
- <https://wiki.gentoo.org/wiki/OpenRC> - 30/06/2025
- <https://busybox.net/> - 01/07/2025
- <https://busybox.net/about.html> - 01/07/2025
- <https://sulincix.gitlab.io/distro-kitabi/> - 05/07/2025
- <https://gitlab.com/turkman/devel/doc/wiki/> - 05/07/2025
- <https://turkman.gitlab.io/devel/doc/wiki/> - 05/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_5c049e44-be57-4652-872f-55def669d917 - 09/07/2025
- <https://www.linuxfromscratch.org/lfs/>
- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)#UEFI_sistemler](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)#UEFI_sistemler) - 08/07/2025
- https://tldp.org/HOWTO/html_single/SquashFS-HOWTO/ - 09/07/2025
- <https://askubuntu.com/questions/437880/extract-a-squashfs-to-an-existing-directory> - 11/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_2ad2ac6b-d067-40b0-9b55-46db0c2c98dc - 10/07/2025
- <https://chatgpt.com/>

Not: Metin düzenlemelerinde chatgpt kullanılmıştır.

Geliştiricilere Mesajımız

Gnu/Linux, açık kaynaklı bir işletim sistemidir. Kullanıcı ve geliştiricilere bir çok avantajlar sunmaktadır. Bunlar;

- Kodlarına erişilebilir(Açık Kaynak)
- Dağıtılabir
- Değişirilebilir
- Virüsten etilenme azdır
- Özelleştirilebilir
- Bağımlılığı azaltır
- Güvenlik en önemli şarttır
- Düşük donanımlarda iyi performan verir

Bu özellikleri açısından Gnu/Linux tercih etmek çok avantajlıdır. Geliştirme aşamalarına destek vermek, öğrenmek bize, toplumumuza ve ülkemize sayısız faydalar saylayacaktır.

Lisans Bilgileri

Bu proje iki farklı lisans altında dağıtılmaktadır:

1. Kaynak Kodlar (Source Code):

Kaynak kodların tamamı, aşağıdaki lisans altında dağıtılmaktadır:

Version 3, 29 June 2007

Copyright (C) <YIL> <İSİM / KURUM>

Bu program özgür bir yazılımdır: Free Software Foundation tarafından yayınlanan GNU Genel Kamu Lisansı'nın (GPL) 3. versiyonu veya (tercihinize bağlı olarak) daha sonraki bir versiyonu altında dağıtılabilir ve/veya değiştirilebilir.

Bu program, yararlı olacağı umuduyla dağıtılmaktadır, ancak **HERHANGİ BİR GARANTİ OLMAKSIZIN**; hatta **SATILABİLİRLİK** veya **BELİRLİ BİR AMACA UYGUNLUK** GARANTİSİ DAHİLİNDE BİLE OLMADAN dağıtılmaktadır. Ayrıntılar için GNU Genel Kamu Lisansı'na bakınız.

Lisansın bir kopyasını şu adresten edinebilirsiniz: <https://www.gnu.org/licenses/gpl-3.0.html>

2. Dokümantasyon ve Medya (Documentation and Media):

Proje içerisindeki tüm dokümantasyon, grafikler, görseller ve metinler aşağıdaki lisans altında dağıtılmaktadır:

Copyright (C) <2025> <İSİM / KURUM>

Bu lisans, aşağıdaki şartlar altında kullanıma izin verir:

- **Atıf (BY):** Bu çalışmayı uygun bir atıf vererek, lisans bağlantısını sağlayarak ve değişiklikleri belirterek paylaşmalısınız.
- **Aynı Lisansla Paylaş (SA):** Bu çalışmadan üretilen türev eserler aynı lisans altında lisanslanmalıdır.

Daha fazla bilgi ve tam lisans metni için şu bağlantıya bakınız:: <https://creativecommons.org/licenses/by-sa/4.0/>

Ek Bilgiler

Bu lisanslar hakkında daha fazla bilgi için:

- GNU GPLv3: <https://www.gnu.org/licenses/gpl-3.0.html>
- CC BY-SA 4.0: <https://creativecommons.org/licenses/by-sa/4.0/>